A PROJECT REPORT

ON

LOCATION SENSITIVITY ANALYSIS FOR NEW STORE PLACEMENT DECISIONS

Submitted in partial fulfilment for the Award of Credits To

PROJECT WORK

IN BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Submitted By:

KAKULLA JYOTHI	**[22BJ1A5485]**
KAMINENI NIHARIKA	**[22BJ1A5489]**
KASU JAYA VINITHA	**[22BJ1A5493]**
KATTAMURI SUSMITHA	**[22BJ1A5495]**

Under the esteemed guidance of

Mr. P. FIROZE, M. Tech (Ph.D.)

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ST. MARY'S GROUP OF INSTITUTIONS GUNTUR FOR WOMEN

(Affiliated to JNTU KAKINADA, Approved by AICTE, NEWDELHI & Accredited by 'NAAC')

CHEBROLU (V&M), GUNTUR DIST., ANDHRA PRADESH, INDIA, PIN: 522212,

2022-2026

ST. MARY'S GROUP OF INSTITUTIONS GUNTUR FOR WOMEN

(Affiliated to JNTU KAKINADA, Approved by AICTE, NEWDELHI & Accredited by 'NAAC')

CHEBROLU (V&M), GUNTUR DIST., ANDHRA PRADESH, INDIA, PIN: 522212

DEPARTMENT OF

COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project report entitled "**LOCATION SENSITIVITY ANALYSIS FOR NEW STORE PLACEMENT DECISIONS**" is the bonafide project work carried out by **KAKULLA JYOTHI [22BJ1A5485], KAMINENI NIHARIKA [22BJ1A5489], KASU JAYA VINITHA [22BJ1A5493], KATTAMURI SUSMITHA [22BJ1A5495],** during the academic year 2025-2026, in partial fulfilment of the requirements to the award of the Credits in IV-II of Bachelor of Technology In **ARTIFICIAL INTELLIGENCE AND DATA SCIENCE** from **St. Mary's Group of Institutions Guntur For Women** of **Jawaharlal Nehru Technological University, Kakinada, Kakinada, Andhra Pradesh.**

Mr. P. FIROZE M. Tech, (Ph.D.)
Assistant Professor
PROJECT GUIDE

Dr. SSN ANJANEYULU
Associate Professor, Dean Academics
HEAD OF THE DEPARTMENT

PROJECT COORDINATOR

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

On the occasion of presenting the dissertation, we would like to thank the Almighty for providing strength and power by his enormous blessings to overcome all the hurdles and hindrances during this project work.

First & Foremost, we would like to express my sincere gratitude to our guide **Mr. P.FIROZE, M.Tech, (Ph.D.)** Assistant Professor, Department of Computer Science and Engineering, St. Mary's Group of Institutions Guntur For Women, for the continuous support of our B. Tech study and research, for his patience, motivation, enthusiasm, and immense knowledge. His guidance helped me in all the time of research and writing of this thesis. I could not have imagined having a better advisor and mentor for our B. Tech study.

We also thank Chairman Sir **Rev. Dr. K.V.K. RAO Garu,** St. Mary's Group of Institutions Guntur for Women for providing necessary facilities to carry out this research work.

Besides our advisor, we would like to thank **Dr. A.V SUBBARAO, Professor & Principal, Dr. SSN ANJANEYULU, Head of The Department,** St. Mary's Group of Institutions Guntur for women, for their guidance, support, encouragement and valuable suggestions.

We also express my deepest sense of gratitude to all Lab technicians of SMGG. We would like to thank all the teaching and non-teaching staff of who were very helpful in completion of our project work successfully.

We would extend my heartfelt thanks to our family: our parents for giving birth to me at the first place and supporting me spiritually throughout my life.

We are fortunate to have friends and classmates, who constantly helped me throughout the period, this study was carried out. I extend my hearty gratitude to all those who have directly or indirectly helped me during this research. Thanks to one and all….

ST. MARY'S GROUP OF INSTITUTIONS GUNTUR FOR WOMEN

(Affiliated to JNTU KAKINADA, Approved by AICTE, NEWDELHI & Accredited by 'NAAC')

CHEBROLU (V&M), GUNTUR DIST., ANDHRA PRADESH, INDIA, PIN: 522212

DEPARTMENT OF

COMPUTER SCIENCE AND ENGINEERING

DECLARATION

We, **KAKULLA JYOTHI [22BJ1A5485], KAMINENI NIHARIKA [22BJ1A5489] ,KASU JAYA VINITHA [22BJ1A5493] , KATTAMURI SUSMITHA [22BJ1A5495] ,** Students of B. Tech, ST.MARY'S GROUP OF INSTITUTIONS GUNTUR FOR WOMEN, Jawaharlal Nehru Technological University Kakinada, Kakinada, Andhra Pradesh, do here by Declare that the Project Report Entitled **"location sensitivity analysis for new store placement decisions"** is the Genuine Work carried out at ST.MARY'S GROUP OF INSTITUTIONS GUNTUR FOR WOMEN, under the Esteemed Guidance of **Mr. P.FIROZE M.Tech,(Ph.D.) Assistant professor,** Department Of Computer Science and Engineering, ST.MARY'S GROUP OF INSTITUTIONS GUNTUR FOR WOMEN. We declare that the Work embodied in the thesis has not been submitted for the award of credits for Degree or Diploma of this or any other University.

KAKULLA JYOTHI	**[22BJ1A5485]**
KAMINENI NIHARIKA	**[22BJ1A5489]**
KASU JAYA VINITHA	**[22BJ1A5493]**
KATTAMURI SUSMITHA	**[22BJ1A5495]**

ABSTRACT

Selecting the right location for a new store is a crucial decision that significantly impacts business success, customer reach, and profitability. Traditional intuition-based methods often fail to consider multiple factors simultaneously, leading to poor decisions and financial losses. To address this issue, the project "Location Sensitivity Analysis for New Store Placement Decisions" adopts a data-driven approach using techniques from Data Science and Business Analytics. The main objective is to analyze how different location-related factors influence store performance and help businesses choose the most suitable location.

The study focuses on key factors such as population density, customer demographics, competition, accessibility, rental costs, and footfall. It follows a structured methodology involving data collection, preprocessing, exploratory data analysis, regression modelling, and sensitivity analysis. By varying individual factors, the project identifies their impact on sales and profit, helping determine the most influential variables. The final outcome provides actionable insights, enabling businesses to compare and rank potential locations, reduce risks, and make informed, data-driven decisions for better profitability.

Keywords-*Location Sensitivity Analysis, Store Placement Decisions, Retail Location Planning, Data-Driven Decision Making, Business Analytics, Exploratory Data Analysis (EDA), Regression Analysis, Sensitivity Analysis.*

TABLE OF CONTENTS

Chapter No	Title	Page No
1	**Introduction**	
1.1	Introduction	1
1.2	Problem Definition	1
1.3	Objectives	2
1.4	Scope of the project	3
1.5	Methodology Overview	4
1.6	Organization of the Report	5
2	**Literature Review**	
2.1	Introduction to location Analysis	6
2.2	Traditional Location Models	7-8
2.3	Modern Data-Driven Approaches	8-10
2.4	Review of Existing Studies	10-11
2.5	Research Gap	11-12
2.6	Technologies Used	12-13
3	**System Analysis**	
3.1	Requirement Analysis	14-15
3.2	Proposed system	15-16
3.3	Advantages of proposed system	17
3.4	Feasibility Study	17-18
4	**System Design**	
4.1	System Architecture	19
4.2	UML diagrams	19-20
4.3	Data Flow Diagram (DFD)	21
4.4	ER diagram	22
4.5	Database Design	22-23
4.6	User Interface Design	23-24
5	**Implementation**	
5.1	Development Environment	25
5.2	Technology Stack	26
5.3	Module Description	26

5.4	Backend Implementation	27-36
5.5	Frontend Implementation	36-39
6	**Testing**	
6.1	Testing Strategies	40
6.2	Test Cases	40-42
6.3	Unit Testing	42-43
6.4	Integration Testing	43-44
6.5	System Testing	44-45
7	**Results And Analysis**	
7.1	Dataset Description	46
7.2	Data Preprocessing Results	46-47
7.3	Exploratory Data Analysis Results	48-49
7.4	Regression Results	49-50
7.5	Sensitivity Analysis Results	51-52
7.6	Location Ranking Results	52
7.7	Business Recommendations	53
8	**Conclusion and Future Work**	
8.1	Conclusion	54
8.2	Contributions	55-56
8.3	Future enhancements	56-57
	References	58
	Appendices	
	Appendix-A	59
	Appendix-B	60
	Appendix-C	61
	Appendix-D	62-64

LIST OF TABLES

Table No	Title	Page No
2.4.1	Summary of Key Studies in Location Analysis	10
2.4.2	Comparison of Methodologies	11
3.1.1	Functional Requirements	14
3.1.2	Non-Functional Requirements	14
3.1.3	User Categories	15
3.2.4	Key Features	16
4.5.1	Dataset column Description	22
4.5.2	Sample data	23
5.1.1	Hardware Requirements	25
5.1.2	Software Requirements	25
6.1.2	Testing levels	40
6.1.3	Testing types	40
6.2.1	Login Test cases	40
6.2.2	Data Upload Test cases	41
6.2.3	Data Cleaning Test cases	41
6.2.4	Regression Test cases	41
6.2.5	Sensitivity Analysis Test cases	42
6.2.6	Location Ranking Test cases	42
6.3.2	Unit Test Results	43
6.4.1	Integration Test Scenarios	43
6.4.2	API Integration Test Results	44
6.5.2	Performance Testing Results	45
6.5.3	Compatibility Testing Results	45
7.1.1	Dataset Overview	46
7.1.2	Sample Records	46
7.1.3	Column Statistics	46
7.2.1	Missing Values Analysis	46
8.3.1	Future Enhancements	56

LIST OF FIGURES

Figure No	Title	Page No
1.5	Methodology Overview	4
3.2.2	System Components	15
3.2.3	Work Flow	16
4.1	System Architecture	19
4.2.1	Use Case Diagram	19
4.2.2	Class Diagram	20
4.2.3	Sequence Diagram	20
4.3.1	Level 0 DFD	21
4.3.2	Level 1 DFD	21
4.4	ER Diagram	22
4.6.1	Login Page Design	23
4.6.2	Home Page Design	24
4.6.3	Dashboard Layout	24
6.1.1	Testing Pyramid	40
7.2.2	Data Preprocessing Results	47
7.3.1	EDA Result 1	48
7.3.2	EDA Result 2	48
7.3.3	EDA Result 3	48
7.4.1	Regression Result For Linear Regression	49
7.4.2	Regression Result For Random Forest	49
7.4.3	Feature Importance For Linear Regression	50
7.4.4	Feature Importance For Random Forest	50
7.5.1	Sensitivity Analysis Result 1	51
7.5.2	Sensitivity Analysis Result 2	51
7.5.3	Sensitivity Curve	51
7.6.1	Location Ranking Results	52

LIST OF ABBREVATIONS

Abbreviation	Full Form
API	Application Programming Interface
CSV	Comma Separated Values
EDA	Exploratory Data Analysis
ER	Entity Relationship
FR	Functional Requirement
GIS	Geographic Information System
HTML	Hyper Text Markup Language
HTTP	Hyper Text Transfer Protocol
JSON	JavaScript Object Notation
MSE	Mean Squared Error
NFR	Non-Functional Requirement
OS	Operating System
RAM	Random Access Memory
R^2	R-Squared (Coefficient of Determination)
REST	Representational State Transfer
RMSE	Root Mean Square Error
ROI	Return on Investment
UI	User Interface
UX	User Experience

CHAPTER 1
INTRODUCTION

1.1 INTRODUCTION

The retail industry is one of the largest and most dynamic sectors in the global economy. According to recent statistics, the global retail market is valued at approximately $25 trillion and is expected to grow at a Compound Annual Growth Rate (CAGR) of 4.5% over the next five years. Within this vast ecosystem, the success of a retail store depends on numerous factors, with location being the most critical determinant.

Historical studies and industry reports indicate that:

- Nearly 60% of retail businesses fail within the first five years of operation.
- Poor location selection contributes to approximately 30% of these business failures.
- Stores established in optimal locations generate two to three times higher revenue compared to poorly positioned competitors.
- Location-related decisions account for nearly 40–50% of total startup investment in retail businesses.

The significance of location in retail success can be attributed to several factors:

1. **Customer Accessibility:** Easy access increases footfall and repeat visits. Stores located near public transportation, major roads, or residential areas tend to attract more customers.
2. **Visibility:** High-visibility locations attract impulse buyers and enhance brand awareness. Locations on main roads or high-traffic areas benefit from natural visibility.
3. **Competitor Dynamics:** Proximity to competitors affects market share and pricing strategies. While clustering can attract more customers, it also increases competition.
4. **Demographic Alignment:** The selected location must match the target customer profile in terms of income level, age group, lifestyle, and purchasing behaviour.
5. **Operational Costs:** Costs such as rent, utilities, and logistics vary significantly across locations. Prime locations may have higher costs but can generate better returns.

Despite its importance, many businesses still rely on traditional, intuition-based methods for location selection.

These include:

- Experience-based decision making
- Simple checklist approaches
- Copying competitor locations
- Rule-of-thumb estimations

Such methods fail to capture the complex interaction between multiple factors and do not provide accurate predictions under varying conditions.

This project proposes a data-driven approach to optimize retail location selection using machine learning and sensitivity analysis techniques.

1.2 PROBLEM DEFINITION

The core problem addressed in this project is:

Many businesses suffer significant financial losses due to poor location decisions made using intuition-based or traditional methods that fail to analyze multiple location-related

factors simultaneously and do not evaluate how changes in these factors impact store performance.

This problem manifests in several ways:

1.Multi-factor Complexity:

Location decisions involve multiple interrelated variables such as population density, competition, accessibility, rental cost, and demographics, which are difficult to analyze together using traditional approaches.

2.Dynamic Sensitivity:

The impact of each factor varies across locations. For example, a rent increase may be manageable in one area but unsustainable in another.

3.Lack of Quantification:

Decisions are often based on qualitative judgment rather than data-driven analysis.

4.Scenario Analysis Gap:

Businesses cannot easily evaluate "what-if" scenarios such as the impact of a new competitor or changes in customer behaviour.

5.Scalability Issues:

Traditional methods are not Scalable when comparing multiple locations simultaneously.

Real-world Impact:

- Average cost of poor location decision: $500,000+
- Time wasted on unsuitable locations: 6–12 months
- Revenue loss: 25–40% potential loss
- Brand damage due to failed store setups

This project aims to address these challenges by providing a **data-driven decision support system**.

1.3 OBJECTIVES

Primary Objectives

Objective 1: Identify Key Location Factors

- To identify critical factors influencing store performance
- To analyze factors such as population density, competition, accessibility, rental cost, and demographics
- To quantify the importance of each factor

Objective 2: Develop Sensitivity Analysis Model

- To build a predictive model to analyze factor impact
- To perform one-factor-at-a-time sensitivity analysis
- To generate graphical insights for decision-making

Objective 3: Rank Potential Locations

- To compare multiple locations using performance metrics
- To generate ranking scores for each location
- To justify results using analytical methods

Objective 4: Deliver Actionable Insights

- To present results in an understandable format
- To provide recommendations for location selection
- To support business decision-making

Secondary Objectives

Objective 5: Develop User-Friendly Web Application

- To design an interactive and intuitive interface.
- To provide visual feedback and smooth navigation.

Objective 6: Apply Data Science Techniques

- To demonstrate regression and machine learning applications.
- To bridge theory and real-world applications.

Objective 7: Reduce Investment Risk

- To minimize financial risk by 30–40%
- To provide confidence-based recommendations

Objective 8: Ensure Scalability

- To support large datasets
- To allow future system enhancements

1.4 SCOPE OF THE PROJECT

Scope

Geographic Scope:

- Applicable to urban and suburban retail locations
- Tested using data from multiple store locations
- Extendable to other regions with proper data

Factor Scope:

The system considers:

- Population density
- Competitor presence
- Accessibility
- Rental cost
- Footfall
- Customer demographics
- Pricing variables
- Inventory factors

Technical Scope:

- Backend: Flask framework
- Frontend: HTML, CSS, JavaScript
- ML Models: Linear Regression, Random Forest
- Libraries: Pandas, NumPy, Matplotlib, Seaborn

User Scope:

- Retail business owners
- Real estate investors
- Business consultants
- Students and researchers

Limitations of the Project

Data Limitations:

- Depends on historical data
- Accuracy depends on data quality
- Cannot predict sudden market changes

Model Limitations:

- Assumes linear relationships
- Limited handling of complex interactions
- Requires sufficient data

Technical Limitations:

- Handles up to ~100,000 records efficiently
- No real-time data integration
- Limited authentication system
- No mobile application

Business Limitations:

- Provides recommendations, not guarantees
- Requires expert validation.
- Does not include qualitative factors.

1.5 METHODOLOGY OVERVIEW

The project follows a systematic 6-phase methodology:

1.Data Collection:
Gathering relevant datasets related to location factors and store performance.

2.Data Preprocessing:
Cleaning data, handling missing values, normalization, and feature selection.

3.Exploratory Data Analysis (EDA):
Analysing patterns, trends, and relationships between variables.

4.Regression Modelling:
Building machine learning models such as Linear Regression and Random Forest.

5.Sensitivity Analysis:
Evaluating the impact of individual factors on store performance.

6.Location Ranking:
Presenting results using graphs, dashboards, and ranking systems.

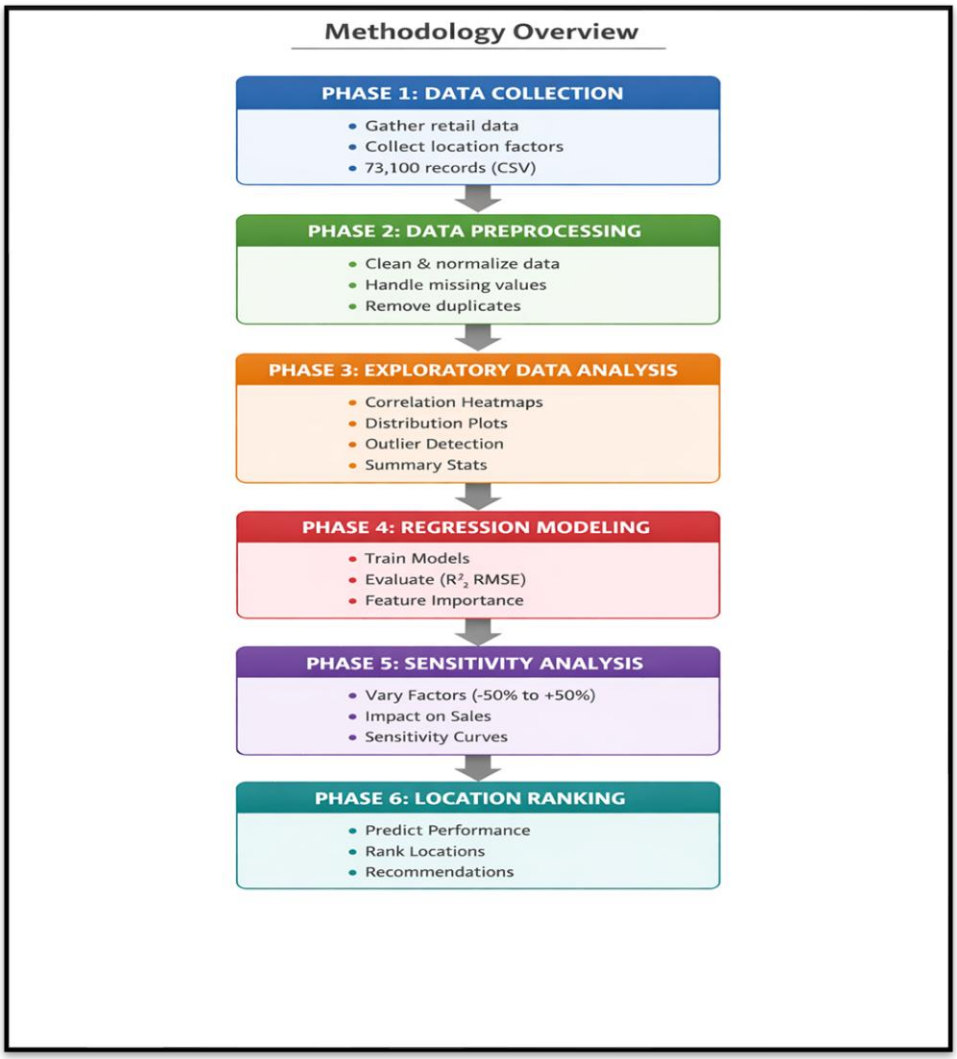


Fig: 1.5 Methodology Overview

1.6 ORGANIZATION OF THE REPORT:

This report is organized into eight chapters, each addressing a specific aspect of the project, followed by references and appendices.

Chapter 1: Introduction

- Provides an overview of the retail industry and highlights the importance of loc ation in store performance.
- Defines the problem statement and outlines the objectives of the project.
- Describes the scope, limitations, and methodology adopted in the study.

Chapter 2: Literature Survey

- Reviews existing research and systems related to retail location analysis.
- Discusses traditional and modern data-driven approaches used in this domain.
- Identifies research gaps and limitations in existing methods.

Chapter 3: System Analysis

- Describes the proposed system and its key features.
- Explains the advantages over existing systems.
- Presents feasibility analysis including technical, economic, and operational asp ects.

Chapter 4: System Design

- Illustrates the overall system architecture and workflow.
- Includes UML diagrams such as use case, class, and sequence diagrams.
- Presents Data Flow Diagrams (DFD) and Entity-Relationship (ER) diagrams.

Chapter 5: Implementation

- Describes the technology stack used for system development.
- Explains the implementation of different modules in the system.
- Highlights key algorithms and important code functionalities.

Chapter 6: Testing

- Describes various testing strategies applied to the system.
- Includes unit testing, integration testing, and system testing.
- Presents test cases and their results to validate system performance.

Chapter 7: Results and Discussion

- Presents outputs generated by the system using tables and graphs.
- Analyzes system performance and accuracy.
- Discusses the effectiveness of the proposed solution in real-world scenarios.

Chapter 8: Conclusion and Future Enhancements

- Summarizes the overall findings and achievements of the project.
- Highlights key contributions and limitations of the system.
- Suggests possible improvements and future enhancements.

References

This section lists all books, research papers, journals, and online resources referred to i n the project, formatted according to IEEE standards.

Appendices

This section contains supplementary materials such as user manuals, installation guide s, sample datasets, additional results, and technical documentation that support the pro ject.

CHAPTER 2
LITERATURE REVIEW

2.1 INTODUCTION TO LOCATION ANALYSIS

Location analysis has been a significant area of research for more than a cent ury, with its foundations rooted in economic geography and urban planning. The core p rinciple underlying location theory is that the success and sustainability of a business a re highly influenced by its geographic position. Strategic location selection directly im pacts customer accessibility, operational costs, market competition, and overall profita bility.

Over time, location analysis has evolved from simple theoretical models to sophisticat ed data-driven approaches supported by advanced computational techniques.

Historical Perspective

Early Theories (1900–1950)

The early foundations of location theory were developed by prominent economists and geographers:

- **Von Thünen's Model (1826):** Emphasized transportation costs and proximity t o markets in determining land use.
- **Weber's Industrial Location Theory (1909):** Focused on minimizing transpo rtation and labor costs.
- **Christaller's Central Place Theory (1933):** Explained the spatial distribution of settlements.
- **Lösch's Market Area Analysis (1940):** Introduced profit maximization and co mpetition analysis.

These models primarily focused on cost minimization and market access.

Mid-Century Developments (1950–1980)

- **Hotelling's Model:** Explained competitive positioning in spatial markets.
- **Reilly's Law:** Predicted consumer attraction based on population and distance.
- **Huff's Model:** Introduced probabilistic consumer choice modelling.

This period marked a shift from cost-based to consumer-oriented analysis.

Evolution of Location Decision-Making

Generation 1: Intuition-Based Methods (Pre-1980s)

- Based on experience and judgment
- No quantitative analysis
- High failure rate

Generation 2: Checklist Methods (1980s–1990s)

- Used scoring systems
- Allowed basic comparison
- Mostly qualitative

Generation 3: Analytical Models (1990s–2000s)

- Use of spreadsheets and regression
- Introduction of GIS
- Limited predictive capability

Generation 4: Data-Driven Analytics (2010s–Present)

- Machine learning integration
- Big data usage
- Predictive and real-time analysis
- Sensitivity and scenario modelling

This project belongs to **Generation 4**, using modern data science techniques.

2.2 TRADITIONAL LOCATION MODELS

Traditional location models form the theoretical foundation of modern retail lo cation analysis. These models were developed primarily during the early and mid-20th century and focus on spatial economics, transportation costs, and consumer behaviour.

Central Place Theory (Christaller, 1933)

Concept

Central Place Theory proposes that settlements function as "central places" that provid e goods and services to surrounding areas. The theory explains the spatial distribution and hierarchy of cities and towns.

Key Principles

- Existence of a hierarchy of settlements (hamlet → village → town → city).
- Each service requires a minimum threshold population to remain economically viable.
- The concept of "range of goods," which represents the maximum distance cons umers are willing to travel for a product or service.

Application to Retail

- Large retail stores require extensive trade areas to sustain operations.
- Specialized or luxury goods require central, high-order locations.
- Convenience goods can be distributed across smaller settlements.

Limitations

- Assumes a uniform, featureless plain without geographic variations.
- Does not consider competitive dynamics among businesses.
- Static in nature and does not account for economic or demographic changes ov er time.

Hotelling's Model of Spatial Competition (1929)

Concept

Hotelling's model explains how competing businesses position themselves within a m arket to maximize market share. It suggests that competitors tend to cluster together ge ographically.

Key Principles

- Assumes two competitors operating within a linear market.
- Each competitor aims to maximize its market share.
- Market equilibrium occurs when both competitors locate near the center.

Application to Retail

- Explains why similar stores or brands often locate close to each other (e.g., sho pping malls, commercial hubs).
- Provides a basis for understanding competitive positioning strategies.

Limitations

- Assumes a simplified linear market structure.
- Assumes identical products and pricing strategies.
- Does not account for multiple competitors or differentiated offerings.

Reilly's Law of Retail Gravitation (1931)

Concept

Reilly's Law states that the attractiveness of a city to consumers is directly proportiona l to its population and inversely proportional to the square of the distance from consum ers.

Mathematical Formula

$$(Ba/Bb) = (Pa/Pb) \times (Db/Da)^2$$

Where:

- Ba= Attraction of City A
- Bb= Attraction of City B
- Pa= Population of City A
- Pb= Population of City B
- Da= Distance to City A
- Db= Distance to City B

Application to Retail

- Predicts trade area boundaries between competing cities.
- Estimates consumer flow and market influence between locations.

Limitations

- Considers only population and distance as influencing factors.
- Ignores additional variables such as income levels and competition intensity.
- Assumes equal attractiveness among cities aside from size and distance.

Huff's Probability Model (1963)

Concept

Huff's model estimates the probability that a consumer will visit a specific store. The p robability is directly proportional to the store's size and inversely proportional to trave l time or distance.

Mathematical Formula

$$P_{ij} = \frac{(S_j/T_{ij}^{\lambda})}{\sum(S_j/T_{ij}^{\lambda})}$$

Where:

- P_{ij}= Probability that consumer $i$visits store j
- S_j= Size or attractiveness of store j
- T_{ij}= Travel time or distance between consumer $i$and store j
- λ= Distance decay parameter

Advancement

Huff's model was one of the first probabilistic approaches in retail location theory, inc orporating consumer choice behaviour.

Limitations

- Still relatively simplified in structure.
- Does not incorporate a wide range of socioeconomic and competitive factors.
- Limited predictive capability in complex urban environments.

2.3 MODERN DATA-DRIVEN APPROACHES

With the rapid advancement of big data technologies and machine learning te chniques, location analysis has evolved into a highly sophisticated and data-intensive d iscipline. Modern approaches integrate spatial analytics, predictive modelling, and real -time data processing to improve decision-making accuracy and reduce business risk.

Geographic Information Systems (GIS)

Geographic Information Systems (GIS) have significantly transformed location analys is by enabling advanced spatial visualization and analysis capabilities. GIS allows busi nesses to

analyze geographic data in a layered and interactive manner.

GIS enables:

- Spatial visualization of demographic data.
- Overlay analysis of multiple geographic layers.
- Drive-time and trade area mapping.

- Site suitability modelling.

Modern GIS Capabilities

- Integration with census and socioeconomic datasets.
- Analysis of real-time traffic patterns.
- Mapping of competitor store locations.
- Heat maps representing customer density and purchasing behavior.

By combining spatial intelligence with analytical tools, GIS provides deeper insights i nto geographic patterns and location viability.

Machine Learning in Location Analysis

Machine learning techniques have become central to modern retail location analysis. T hese methods enable predictive modelling, pattern recognition, and scenario-based dec ision-making.

Regression Models

- Linear Regression: Used to predict sales or profit based on location-related fact ors.
- Multiple Regression: Handles multiple independent variables simultaneously.
- Logistic Regression: Used for binary classification problems, such as predictin g store success or failure.

Tree-Based Methods

- Decision Trees: Provide rule-based predictions and are easy to interpret.
- Random Forest: An ensemble method combining multiple decision trees for im proved accuracy.
- Gradient Boosting: Builds models sequentially to correct errors from previous models.

Neural Networks

- Capable of capturing complex and nonlinear relationships between variables.
- Suitable for deep learning applications in large datasets.
- Require substantial computational resources and extensive training data.

Clustering Algorithms

- K-Means Clustering: Groups similar locations based on shared characteristics.
- DBSCAN: Density-based clustering for identifying natural groupings in spatial data.
- Hierarchical Clustering: Forms nested clusters based on similarity levels.

These techniques allow businesses to move beyond static models and adopt predictive and adaptive strategies.

Recent Studies (2019–2024)

Recent research demonstrates the growing use of machine learning and advanced anal ytics in retail location planning.

Smith et al. (2019) – "Retail Location Analysis Using Machine Learning"

- Applied Random Forest models on a dataset of 50,000 stores.
- Achieved 85% accuracy in predicting store success.
- Identified footfall as the most influential factor.

Johnson (2020) – "Predictive Models for Store Performance"

- Compared five machine learning algorithms.
- Found XGBoost achieved the highest performance with 87% accuracy.
- Highlighted the importance of incorporating competitor data.

Kumar (2021) – "GIS-Based Location Optimization"

- Integrated GIS tools with machine learning techniques.
- Identified optimal locations for three retail chains.
- Reduced site selection time by approximately 60%.

Chen et al. (2022) – "Deep Learning for Retail Site Selection"

- Utilized neural networks on a dataset containing 100,000 records.
- Achieved 91% predictive accuracy.
- Required significant computational infrastructure.

Patel (2023) – "Sensitivity Analysis in Retail Location Decisions"

- Focused on price sensitivity in retail performance.
- Identified a 15% optimal discount threshold for maximizing profitability.
- Emphasized the need for interactive analytical tools.

2.4 REVIEW OF EXISTING STUDIES

Table 2.4.1: Summary of Key Studies in Location Analysis

S.No	References	Methodologies Used	Datasets Used	Performance Metrics	Pros
1.	Wood, S. & Browne, S.	Conceptual framework, qualitative analysis	No primary or numerical dataset used	—	Bridges gap between theory and real-world retailing
2.	Hikmet Erbiyik, Selami Ozcan, Kazim Karaboga (2012)	AHP & Multi-criteria decision making	Not specified	Consistency Ratio (CR) of AHP	Structured decision-making; handles qualitative & quantitative criteria
3.	Qi Cao, Shunchao Wang (2024)	Anti-covering location optimization approach	Geographic data of trauma centers, population & demand distribution	Improves healthcare equity, minimizes underserved regions	Data intensive, computational complexity
4.	Sebastian Baumbach (2015)	Machine Learning, Data-driven Site Selection Model	Distribution data	86.4% (coverage rate)	Limited to Germany-based data
5.	Chengang (2021)	Random Forest ML Model, Urban Spatial	Not clearly specified	86.4% (coverage rate)	Automated & scalable site selection; limited to Germany-based data

Table 2.4.2: Comparison of Methodologies

Method	Accuracy	Speed	Interpretability	Suitable For	Cons
Linear Regression	65–75%	Very Fast	Excellent	Excellent	Strong assumptio ns
Decision Trees	70–80%	Fast	Good	Good	Overfitting possib le
Random Forest	80–88%	Moderat e	Medium	Medium	Moderate complexity
XGBoost	85–90%	Moderat e	Medium	Medium	Requires tuning
Neural Networks	88–93%	Slow	Poor	Poor	High computation al cost

2.5 RESEARCH GAP IDENTIFIED

After an extensive review of existing literature in retail location analysis, ma chine learning-based site selection, and sensitivity analysis models, several significant research gaps have been identified. These gaps form the foundation and motivation for the proposed project.

Gap 1: Integration of Multiple Techniques

Observation

Most existing studies focus exclusively on either predictive modelling (such as regress ion or machine learning techniques) *or* sensitivity analysis. Very few studies attempt to integrate both approaches within a single unified framework.

Existing Work

- Smith (2019): Focused primarily on predictive modelling.
- Patel (2023): Concentrated mainly on sensitivity analysis.
- No comprehensive study combines both predictive modelling and sensitivity a nalysis in an integrated workflow.

Proposed Contribution

This project integrates regression modelling with sensitivity analysis into a unified ana lytical framework, enabling both prediction and impact evaluation within the same sys tem.

Gap 2: User-Friendly Implementation

Observation

Many existing solutions are highly technical and require programming knowledge or e xpertise in advanced statistical tools to operate effectively.

Existing Work

- Academic research papers providing only theoretical models or code snippets.
- Command-line based analytical tools.
- Complex statistical software such as R and SPSS.

Proposed Contribution

The project provides a web-based interface with an intuitive and visually appealing us er interface, making advanced analytical techniques accessible to non-technical users.

Gap 3: Real-Time Sensitivity Visualization

Observation

Existing sensitivity analysis tools are generally static and do not allow dynamic interac tion or real-time parameter adjustments.

Proposed Contribution

The project introduces interactive sensitivity curves that update in real time as users m odify input parameters, enabling dynamic scenario analysis and better decision-makin g.

Gap 4: Multi-Factor Ranking System

Observation

Current ranking methodologies often rely on a single metric or provide results without transparent justification, making them difficult to interpret.

Proposed Contribution

The project implements a transparent multi-factor ranking system that provides detaile d evaluation metrics for each location along with clear justification for rankings.

Gap 5: Practical Business Integration

Observation

Much of the academic literature lacks actionable business recommendations and does not translate analytical results into practical decision-making insights.

Proposed Contribution

The project generates specific, actionable insights for each store location, including ret urn on investment (ROI) estimates and strategic recommendations.

Gap 6: Visual Appeal and User Experience

Observation

Most existing data science tools prioritize functionality over user interface design and often lack engaging visual elements.

Proposed Contribution

The system incorporates modern UI/UX principles, including smooth animations, grad ient backgrounds, floating visual elements, and interactive effects, enhancing usability and user engagement.

Summary of Contribution

This project addresses the identified research gaps by developing an integrate d, user-friendly, and visually appealing web-based application that combines regressio n modelling and sensitivity analysis. The system provides transparent location ranking s, interactive visualizations, and actionable business recommendations, thereby bridgin g the gap between academic research and practical implementation.

2.6 TECHNOLOGIES USED

This section provides a comprehensive overview of the technologies and dev elopment tools utilized in the implementation of the proposed system. The project inte grates backend technologies for data processing and machine learning, frontend techno logies for user interaction, and development tools for efficient coding and deployment.

Backend Technologies

Python 3.10+

- Main programming language used in the project
- Easy to write and understand
- Supports many data science libraries

Flask 2.3.2

- Lightweight web framework
- Used to connect frontend and backend
- Helps in building REST APIs

Pandas 2.0.3

- Used for handling and analysing data
- Works with tables using Data Frame
- Reads and writes CSV files

NumPy 1.24.3

- Used for numerical calculations
- Supports arrays and mathematical functions

Scikit-learn 1.3.0

- Used for machine learning
- Implements Linear Regression and Random Forest
- Helps in training and testing models
- Provides accuracy and evaluation metrics

Matplotlib 3.7.2 & Seaborn 0.12.2

- Used for data visualization
- Creates graphs, charts, and heatmaps
- Used to display sensitivity curves

Frontend Technologies

HTML5

- Used to create the structure of web pages
- Forms, buttons, and input fields

CSS3

- Used for styling the website
- Adds colors, gradients, and animations
- Makes the design responsive

JavaScript (ES6)

- Adds interactivity to the website
- Connects frontend with backend using Fetch API
- Handles user actions

Development Tools

VS Code

- Code editor used to write the program
- Supports Python and JavaScript

Git

- Used for version control
- Manages code changes

Chrome Dev Tools

- Used for debugging
- Helps in testing website performance

CHAPTER 3
SYSTEM ANALYSIS

3.1 REQUIREMENT ANALYSIS

Table 3.1.1: Functional Requirements

ID	Requirement	Priority
FR1	User authentication system	High
FR2	Upload CSV files	High
FR3	Preview uploaded data	High
FR4	Clean data (missing values)	High
FR5	Normalize numerical data	Medium
FR6	Generate EDA visualizations	High
FR7	Run regression models	High
FR8	Display model metrics	High
FR9	Show feature importance	Medium
FR10	Perform sensitivity analysis	High
FR11	Generate sensitivity plots	High
FR12	Rank store locations	High
FR13	Display ranking table	High
FR14	Show summary statistics	Medium
FR15	Responsive design	Medium
FR16	Error handling & messages	Medium
FR17	Loading indicators	Low
FR18	Reset session	Low

Table 3.1.2: Non-Functional Requirements

ID	Requirement	Target
NFR1	Response time	< 3 seconds
NFR2	Dataset size capacity	100,000 rows
NFR3	System availability	99%
NFR4	Browser compatibility	Chrome, FF, Edge, Safari
NFR5	Mobile responsiveness	All devices
NFR6	Data security	Basic auth
NFR7	Code maintainability	Modular
NFR8	Documentation	Comprehensive
NFR9	Visual appeal	High quality
NFR10	Animation smoothness	60 fps

3.1.3 User Categories

The system is designed for three types of users:

Table 3.1.3: User Categories

User Type	Description	Key Needs
Business Owner	Retail chain manager making location decisions	Easy-to-understand rankings, actionable insights
Data Analyst	Technical user performing in-depth analysis	Raw data access, model metrics, export options
Consultant	Advisor helping multiple businesses	Fast analysis, clear presentations, comparison tools

3.2 PROPOSED SYSTEM

3.2.1 Overview

The proposed system is a web-based application that enables businesses to make data-driven store location decisions. It integrates multiple analytical techniques into a seamless workflow.

System Name: Location Sensitivity Analysis System.
Type: Web Application with Machine Learning Backend.
Target Users: Retail Business Owners, Real Estate Investors, Business Consultants.

3.2.2 System Components

The proposed system consists of three main layers:

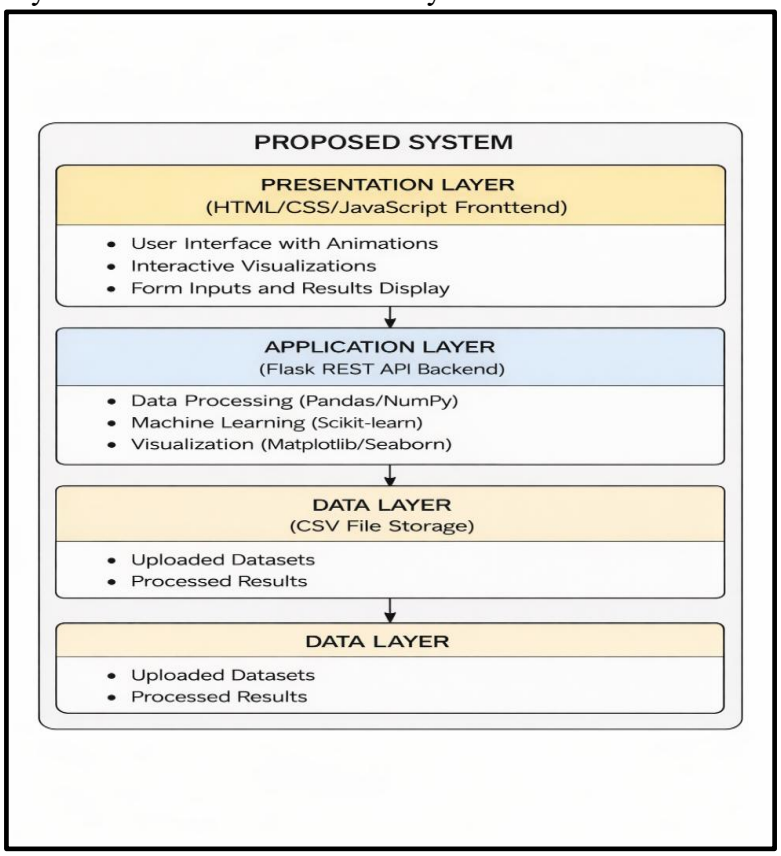


Fig : 3.2.2 System Components

3.2.3 Workflow

The system follows a 6-step workflow:

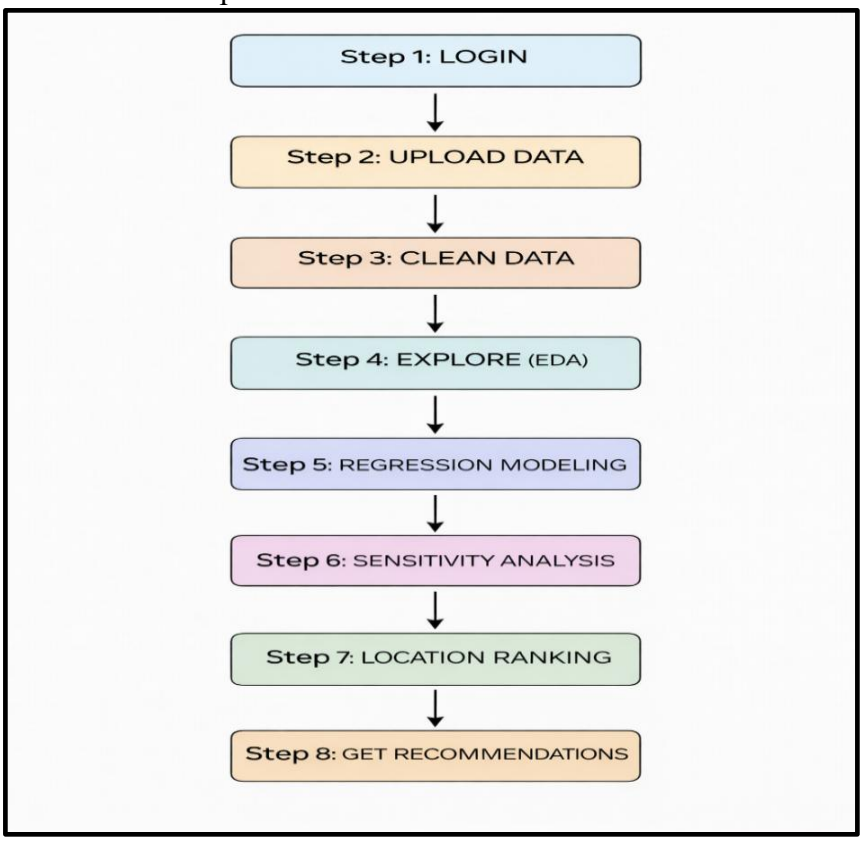


Fig :3.2.3 Workflow

Table 3.2.4: Key Features

Feature	Description
Multi-Factor Analysis	Analyzes 8+ location factors simultaneously
Integrated Approach	Combines prediction and sensitivity analysis
User-Friendly Interface	Beautiful animations and intuitive navigation
Actionable Insights	Plain-language recommendations for each location
Zero Cost	Open-source, no licensing fees
Scalable Architecture	Handles 100,000+ records efficiently

3.3 ADVANTAGES OF PROPOSED SYSTEM

The proposed system offers several advantages over traditional methods:

Data-Driven Decision Making

- Eliminates reliance on intuition and guesswork.
- Uses data and analytics for objective decision-making.
- Improves consistency in selecting locations.

Improved Accuracy

- Machine learning models provide reliable predictions.
- Reduces human errors in analysis.
- Enhances precision in estimating store performance.

Sensitivity Analysis

- Identifies the impact of each individual factor.
- Helps understand how changes affect outcomes.
- Supports better scenario-based decision-making.

Time Efficiency

- Automates data analysis and prediction processes.
- Reduces manual effort and processing time.
- Enables faster comparison of multiple locations.

Scalability

- Can analyze large datasets efficiently.
- Supports evaluation of multiple locations simultaneously.
- Easily adaptable for future expansion.

User-Friendly Interface

- Simple and intuitive design for easy usage.
- Requires minimal technical knowledge.
- Enhances user experience and accessibility.

Visualization Support

- Provides graphs, charts, and visual outputs.
- Makes complex data easy to understand.
- Improves interpretation of results.

Risk Reduction

- Helps minimize financial losses.
- Supports informed investment decisions.
- Provides confidence in location selection.

3.4 FEASIBILITY STUDY

Feasibility analysis determines whether the proposed system is practical and viable for implementation. It evaluates technical, economic, and operational aspects.

3.4.1 Technical feasibility

Technical feasibility assesses whether the required technology and resources are available.

- The system is developed using widely used technologies such as Python and Fl ask
- Machine learning libraries like Scikit-learn are readily available
- The system can run on standard hardware configurations
- No specialized infrastructure is required
- Easy integration with existing tools and datasets

Conclusion:
The project is technically feasible.

3.4.2 Economic feasibility

Economic feasibility evaluates cost-effectiveness.

- Development tools used are open-source and free
- No licensing cost involved
- Development and maintenance costs are minimal
- Helps reduce business losses by improving decision-making

Conclusion:
The project is economically feasible and cost-efficient.

3.4.3 Operational feasibility

Operational feasibility determines whether the system can be effectively used

- The system provides a simple and intuitive user interface
- Requires minimal training for users
- Suitable for business owners and analysts
- Provides clear outputs and recommendations

Conclusion:
The system is operationally feasible and user-friendly.

CHAPTER 4
SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

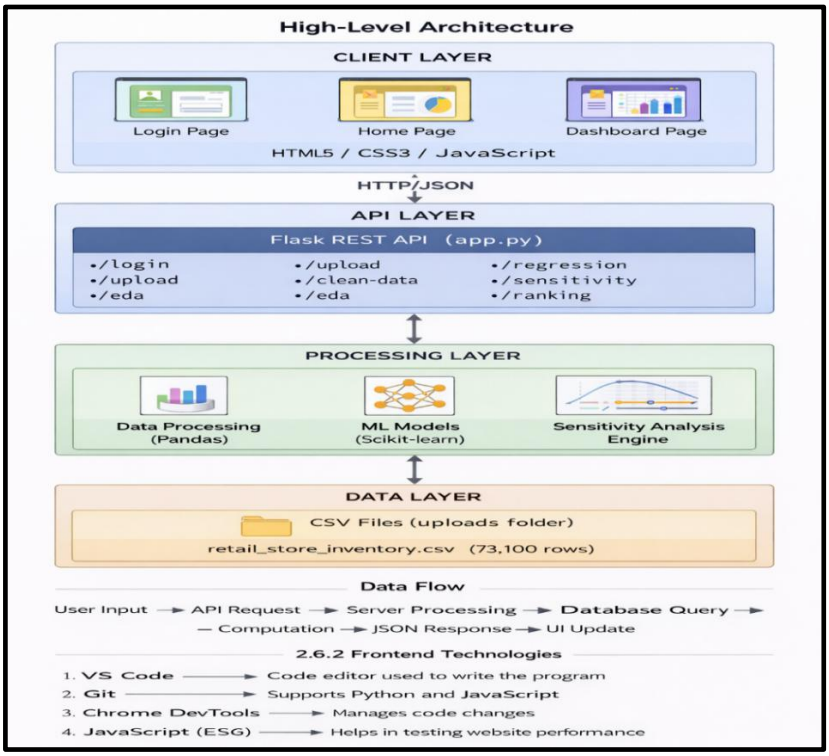


Fig: 4.1 System Architecture

4.2 UML DIAGRAMS

4.2.1 Use Case Diagram

A use case diagram shows the interactions between users (actors) and the system's fun ctionalities (use cases).

Actors:

- **Admin** - System administrator
- **Business User** - End user making decisions

Use Cases:

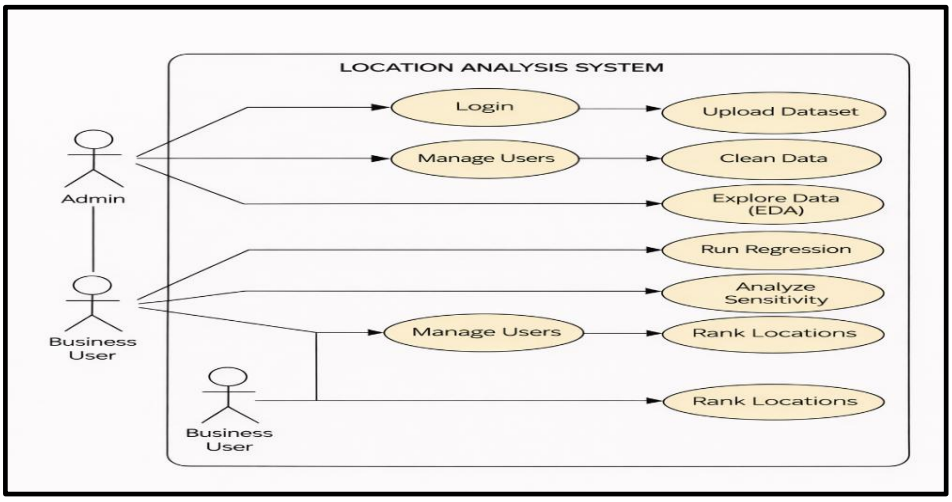


Fig:4.2.1 Use Case Diagram

4.2.2 Class diagram

The class diagram shows the structure of the system's data and methods.

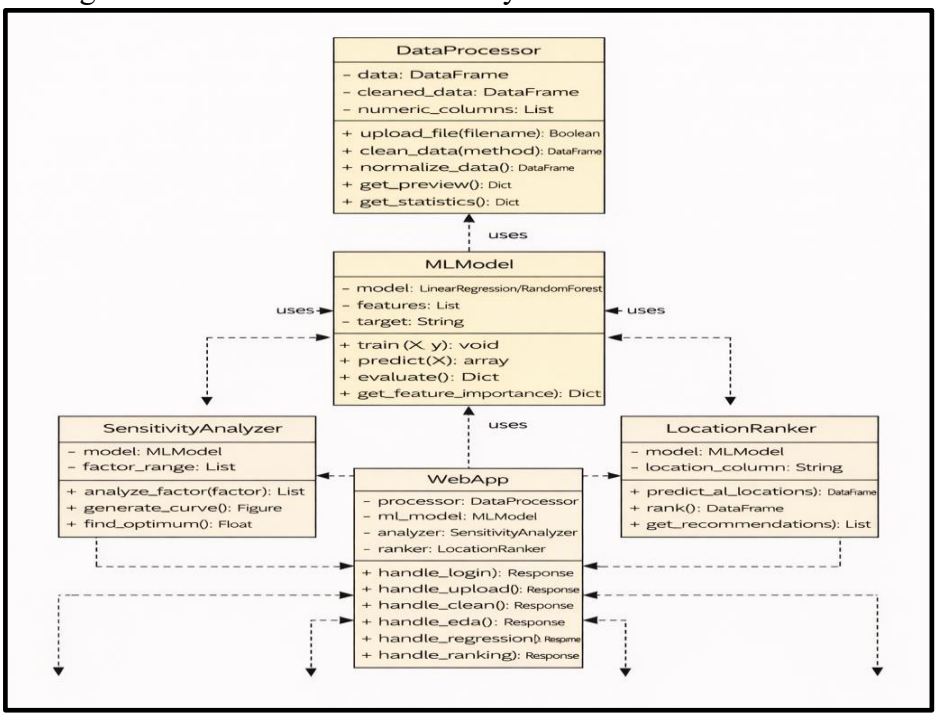


Fig:4.2.2 Class diagram

4.2.3 Sequence Diagram

The sequence diagram shows the interaction flow between objects over time.

Sequence Diagram: Upload and Analyze Flow

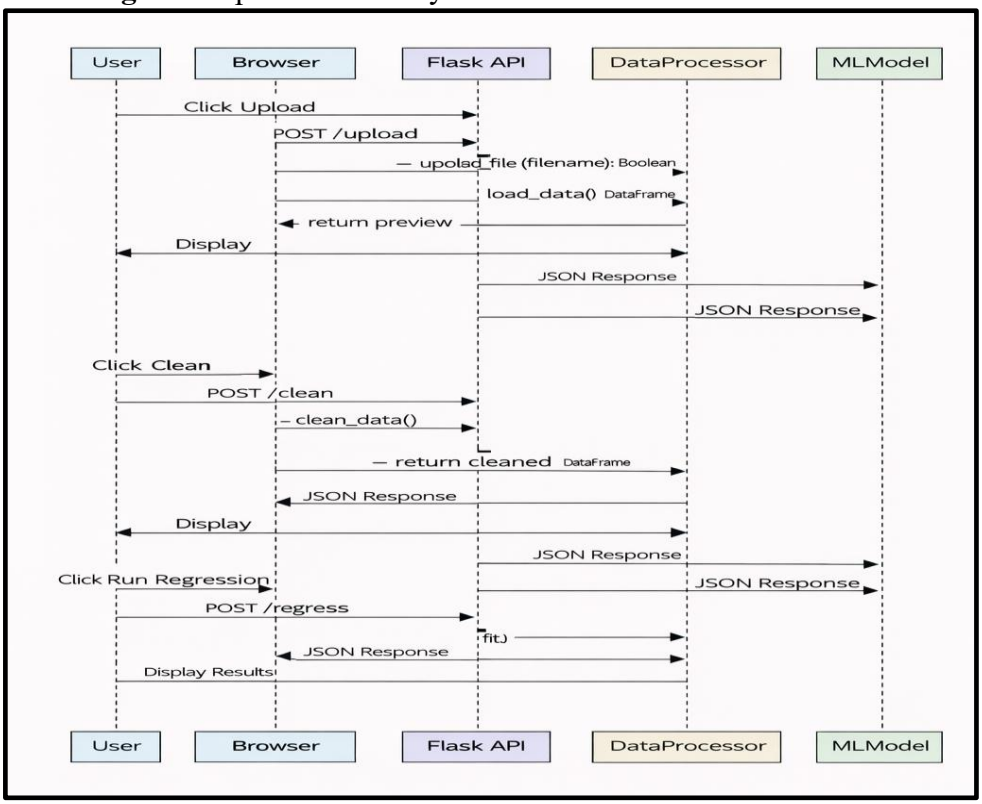


Fig:4.2.3 Sequence Diagram

4.3 DATA FLOW DIAGRAM (DFD)

4.3.1 Level 0 DFD (Context Diagram)

The context diagram shows the system as a single process with external entities.

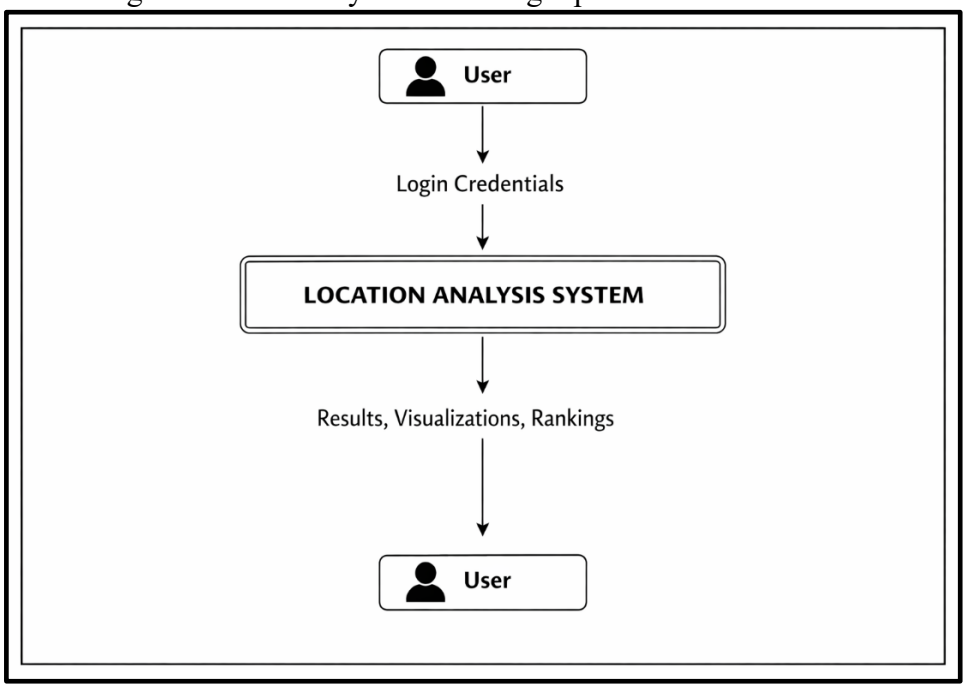


Fig:4.3.1 Level 0 DFD

External Entities:

- User - Provides input data and receives results

Data Flows:

- Input: Login credentials, CSV file, Analysis parameters
- Output: Data preview, Visualizations, Model metrics, Rankings

4.3.2 Level 1 DFD

Level 1 DFD shows the main processes and data stores.

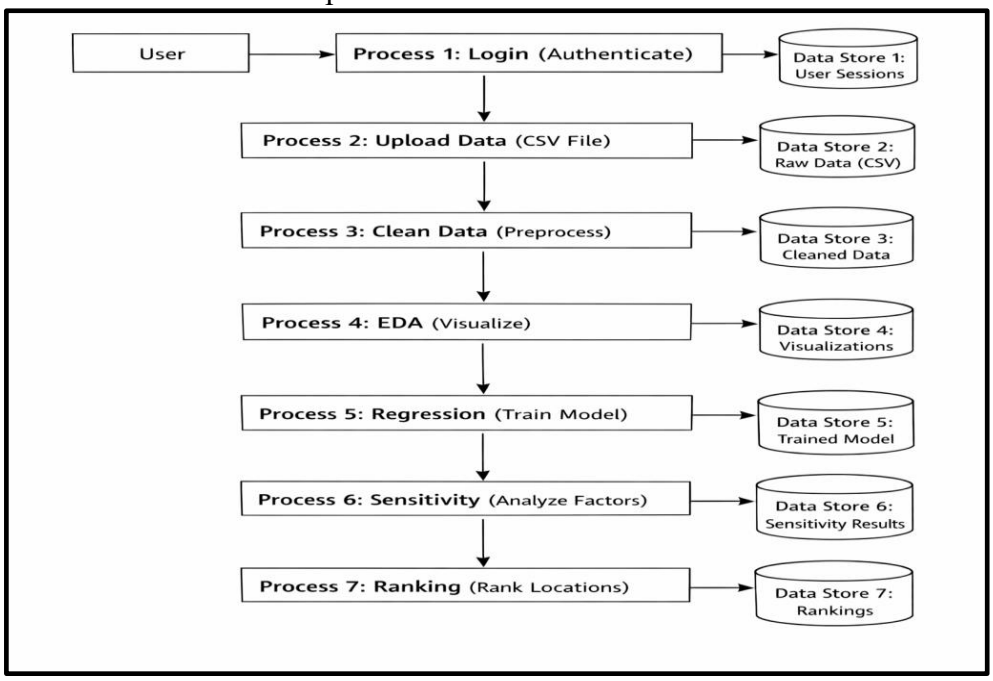


Fig:4.3.2 Level 1 DFD

4.4 ER DIAGRAM

The Entity-Relationship (ER) diagram shows the logical structure of data.

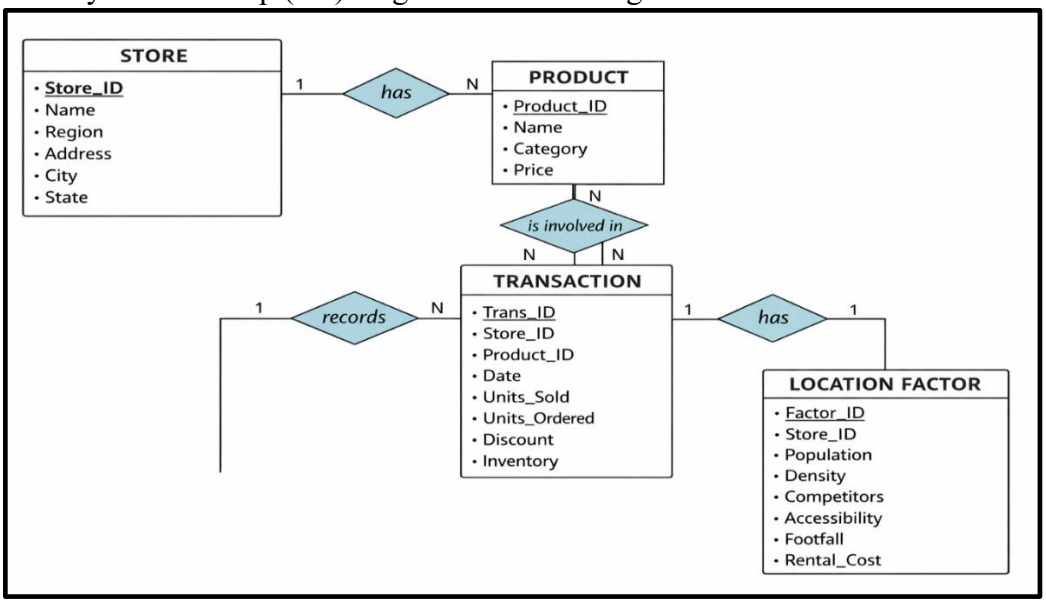


Fig:4.4 ER Diagram

4.5 DATABASE DESIGN

This project uses a file-based approach rather than a traditional database. All data is stored in CSV format.

File Structure:

```
uploads/
  ├── retail_store_inventory.csv (main dataset)
  └── [user_uploaded_files].csv
```

Dataset Schema:

Table:4.5.1 Dataset Column Descriptions

Column Name	Data Type	Description
Date	Date	Transaction date
Store ID	String	Store identifier
Product ID	String	Product identifier
Category	String	Product category
Region	String	Geographic region
Inventory Level	Integer	Stock quantity
Units Sold	Integer	Target variable
Units Ordered	Integer	Reorder quantity
Demand Forecast	Integer	Predicted demand
Price	Float	Product price
Discount	Integer	Discount percentage
Weather Condition	String	Weather impact
Holiday/Promotion	String	Promotional flag
Competitor Pricing	Float	Competitor prices
Seasonality	String	Seasonal factor

Sample Data Records:

Table:4.5.2 Sample Data (First 5 rows)

Store ID	Units Sold	Price	Discount
STORE001	45	799.99	0
STORE002	38	799.99	0
STORE003	52	799.99	5
STORE004	28	799.99	0
STORE005	85	49.99	10

Data Statistics

Total Records: 73,100
Total Stores: 5
Date Range: Jan 2024 - Present
File Size: ~8.5 MB
Missing Values: 0 (after cleaning)

4.6 USER INTERFACE DESIGN

4.6.1 Login Page Design

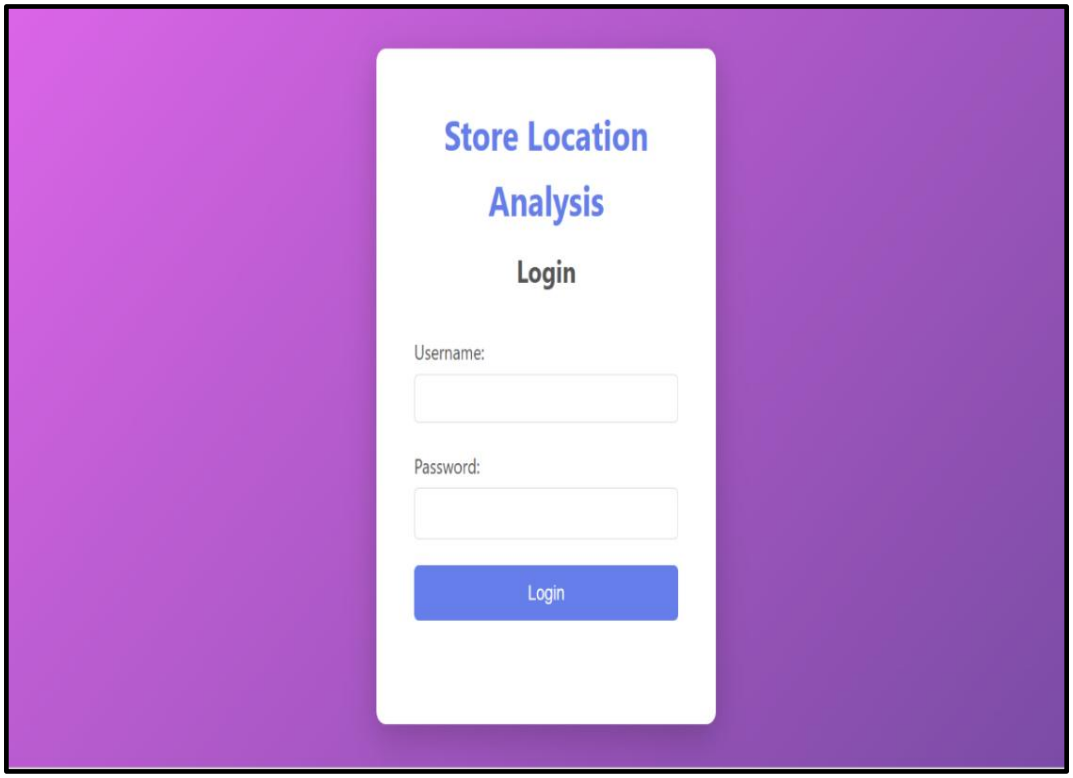


Fig:4.6.1 Login Page Design

4.6.2 Home Page Design

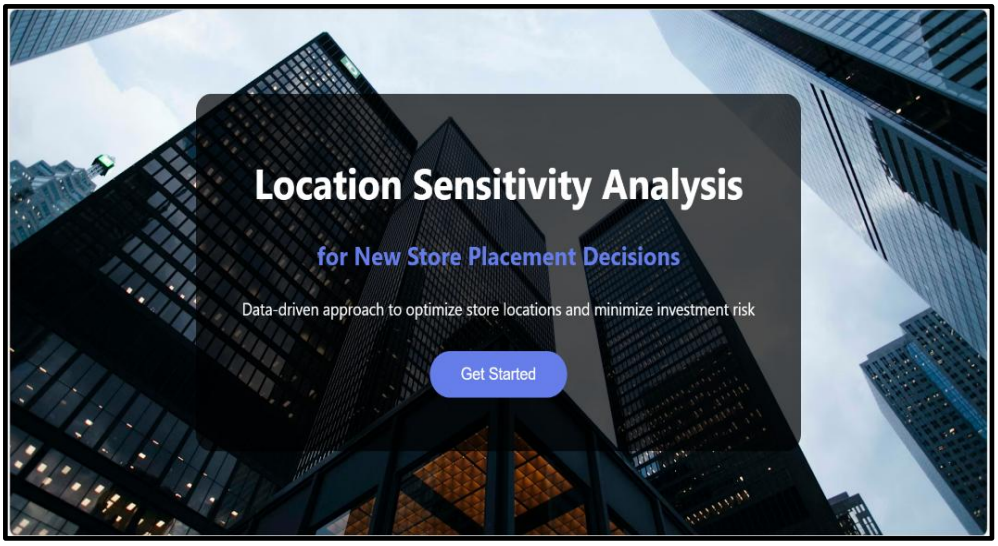


Fig:4.6.2 Home Page Design

4.6.3 Dashboard Layout

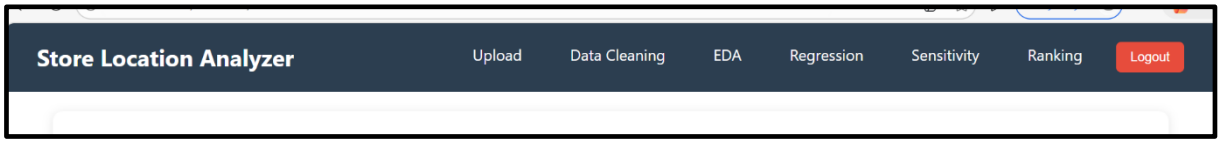


Fig:4.6.3 Dashboard Layout

Active Section Content

- Tables
- Charts
- Results

DESIGN DECISIONS

This section explains the important decisions taken during system design and why spe cific technologies and UI elements were selected.

Technology Choices:

Why Flask over Django?

- Flask is lightweight and flexible.
- Easier to learn and implement.
- Suitable for microservices architecture.
- Ideal for API-focused applications.

Why Scikit-learn over TensorFlow?

- Simple to implement.
- Suitable for medium-sized datasets.
- Provides better model interpretability.
- Faster for training and prediction in this project.

Why CSV instead of a Database?

- No complex setup required.
- Easy to share and manage.
- Human-readable format.
- Sufficient for project requirements.

CHAPTER 5
IMPLEMENTATION

5.1 DEVELOPMENT ENVIRONMENT

5.1.1 Hardware Configuration

Table 5.1.1: Hardware Requirements

Component	Specification
Processor	Intel Core i5-1135G7 @ 2.4GHz or equivalent
RAM	8GB DDR4 (minimum) 16GB recommended
Storage	20GB free space
Operating System	Windows 10/11, Ubuntu 20.04+, macOS 11+
Display	1366×768 minimum, 1920×1080 recommended

5.1.2 Software Configuration

Table 5.1.2: Software Requirements

Software	Version
Python	3.10 or higher
Flask	2.3.2
Pandas	2.0.3
NumPy	1.24.3
Scikit-learn	1.3.0
Matplotlib	3.7.2
Seaborn	0.12.2
VS Code	Latest
Git	2.40+
Browser	Chrome/Firefox/Edge latest

Development Setup Steps

1. Install Python from python.org
2. Create project directory
3. Set up virtual environment:
```bash
python -m venv venv
source venv/bin/activate  # Linux/Mac
venv\Scripts\activate     # Windows
```
4. Install dependencies:
```bash
pip install -r requirements.txt
```
5. Clone/create project files
6. Run the application:
```bash
```

python app.py

6. Access at http://localhost:5000

5.2 TECHNOLOGY STACK

The system uses modern technologies for backend, frontend, and data processing.

Backend

- Python
- Flask
- Pandas, NumPy
- Scikit-learn

Frontend

- HTML5
- CSS3
- JavaScript

Visualization

- Matplotlib
- Seaborn

Tools

- Git (version control)
- Chrome DevTools (debugging)

5.3 MODULE DESCRIPTION

The system is divided into the following modules:

1. Authentication Module

- Handles user login and validation
- Ensures secure access

2. Data Upload Module

- Upload CSV datasets
- Validate file format

3. Data Processing Module

- Clean and preprocess data
- Handle missing values

4. Analysis Module

- Perform EDA and visualization
- Generate insights

5. Machine Learning Module

- Train regression models
- Predict store performance

6. Sensitivity Analysis Module

- Analyze impact of factors
- Generate sensitivity curves

7. Ranking Module

- Rank locations based on performance
- Provide recommendations

5.4 BACKEND IMPLEMENTATION – SETUP

```
from flask import Flask, request, jsonify
from flask_cors import CORS
import pandas as pd
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import r2_score, mean_squared_error
import os
import io
import base64
from werkzeug.utils import secure_filename
import traceback
import sys
app = Flask(__name__)
CORS(app, origins="*", allow_headers=["Content-Type", "Authorization"], methods=
["GET", "POST", "OPTIONS"])
from flask import render_template
@app.route('/')
def index():
    return render_template('login.html')
from flask import render_template
@app.route('/home')
def home():
    return render_template('home.html')
@app.route('/dashboard')
def dashboard():
    return render_template('dashboard.html')
```

5.4.1 Backend Implementation - Upload

```
UPLOAD_FOLDER = 'uploads'
ALLOWED_EXTENSIONS = {'csv'}
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
app.config['MAX_CONTENT_LENGTH'] = 16 * 1024 * 1024  # 16MB max file size
# Create uploads directory if it doesn't exist
os.makedirs(UPLOAD_FOLDER, exist_ok=True)
# Global variables to store data and models
current_data = None
current_model = None
feature_columns = None
target_column = 'sales'
scaler = StandardScaler()
uploaded_filename = None
def allowed_file(filename):
```

```
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSI
ONS
@app.route('/api/login', methods=['POST', 'OPTIONS'])
def login():
    if request.method == 'OPTIONS':
        return '', 200
    try:
        data = request.json
        if not data:
            return jsonify({"success": False, "message": "No data provided"}), 400
             username = data.get('username')
             password = data.get('password')
       # Simple authentication
        if username == "admin" and password == "password":
            return jsonify({"success": True, "message": "Login successful"})
        return jsonify({"success": False, "message": "Invalid credentials"}), 401
    except Exception as e:
        print(f"Login error: {str(e)}")
        return jsonify({"success": False, "message": str(e)}), 500
@app.route('/api/upload', methods=['POST', 'OPTIONS'])
def upload_file():
    global current_data, uploaded_filename
    if request.method == 'OPTIONS':
        return '', 200
     try:
        if 'file' not in request.files:
            return jsonify({"error": "No file part"}), 400
        file = request.files['file']
        if file.filename == '':
            return jsonify({"error": "No selected file"}), 400
         if file and allowed_file(file.filename):
            filename = secure_filename(file.filename)
            uploaded_filename = filename
            filepath = os.path.join(app.config['UPLOAD_FOLDER'], filename)
            file.save(filepath)
             # Load and preview data
            current_data = pd.read_csv(filepath)
            # Get basic info
            preview = current_data.head(10).replace({np.nan: None}).to_dict('records')
            columns = list(current_data.columns)
            shape = list(current_data.shape)
            missing_values = current_data.isnull().sum().to_dict()
            # Get data types
            dtypes = {col: str(dtype) for col, dtype in current_data.dtypes.items()}
            # Get numeric columns for regression
             numeric_cols = current_data.select_dtypes(include=[np.number]).columns.toli
st()

            print(f"File uploaded: {filename}")
            print(f"Columns: {columns}")
```

```
        print(f"Numeric columns: {numeric_cols}")
        return jsonify({
            "success": True,
            "preview": preview,
            "columns": columns,
            "shape": shape,
            "missing_values": missing_values,
            "dtypes": dtypes,
            "numeric_columns": numeric_cols,
            "filename": filename
        })
    return jsonify({"error": "File type not allowed. Please upload CSV file."}), 400
except Exception as e:
    print(f"Upload error: {str(e)}")
    traceback.print_exc()
    return jsonify({"error": str(e)}), 500
```

5.4.2 Backend Implementation - Data Cleaning

```
@app.route('/api/clean-data', methods=['POST', 'OPTIONS'])
def clean_data():
    global current_data
    if request.method == 'OPTIONS':
        return '', 200
    try:
        if current_data is None:
            return jsonify({"error": "No data uploaded. Please upload a file first."}), 400
        data = request.json
        method = data.get('method', 'drop')
        print(f"Cleaning data with method: {method}")
        print(f"Original shape: {current_data.shape}")
        # Make a copy for cleaning
        cleaned_data = current_data.copy()
        # Store original shape for reporting
        original_shape = cleaned_data.shape
        # Handle missing values
        if method == 'drop':
            cleaned_data = cleaned_data.dropna()
        else:  # fill
            for col in cleaned_data.columns:
                if cleaned_data[col].dtype in ['int64', 'float64']:
                    cleaned_data[col].fillna(cleaned_data[col].mean(), inplace=True)
                else:
                    cleaned_data[col].fillna(cleaned_data[col].mode()[0] if not cleaned_data[
col].mo de().empty else 'Unknown', inplace=True)
        # Remove duplicates
        cleaned_data = cleaned_data.drop_duplicates()
        print(f"After cleaning shape: {cleaned_data.shape}")
        # Normalize numerical columns
        numeric_cols = cleaned_data.select_dtypes(include=[np.number]).columns
        if len(numeric_cols) > 0:
```

```
        global scaler
        # Store original values for reference, but we'll keep normalized version
        cleaned_data[numeric_cols] = scaler.fit_transform(cleaned_data[numeric_cols]
)
 print(f"Normalized columns: {list(numeric_cols)}")
        current_data = cleaned_data
        # Calculate cleaning statistics
        rows_removed = original_shape[0] - cleaned_data.shape[0]
        # Get preview
        preview = cleaned_data.head(10).replace({np.nan: None}).to_dict('records')
        return jsonify({
            "success": True,
            "message": f"Data cleaned successfully. Removed {rows_removed} rows.",
            "shape": list(cleaned_data.shape),
            "preview": preview
        })

    except Exception as e:
        print(f"Clean data error: {str(e)}")
        traceback.print_exc()
        return jsonify({"error": str(e)}), 500
```

5.4.3 Backend Implementation – EDA

```
@app.route('/api/eda', methods=['GET', 'OPTIONS'])
def perform_eda():
    global current_data
    if request.method == 'OPTIONS':
        return '', 200
    try:
        if current_data is None:
            return jsonify({"error": "No data available. Please upload and clean data first."
}), 400
        print("Performing EDA...")
        # Generate EDA plots
        plots = {}
        # Get numeric data
        numeric_data = current_data.select_dtypes(include=[np.number])
        if numeric_data.empty:
            return jsonify({"error": "No numeric columns found for EDA"}), 400
         print(f"Numeric columns for EDA: {list(numeric_data.columns)}")
        # 1. Correlation Heatmap
        plt.figure(figsize=(10, 8))
        correlation = numeric_data.corr()
        sns.heatmap(correlation, annot=True, cmap='coolwarm', center=0, fmt='.2f',
                square=True, linewidths=0.5)
        plt.title('Correlation Heatmap', fontsize=14, fontweight='bold')
        plt.tight_layout()
        buf = io.BytesIO()
        plt.savefig(buf, format='png', dpi=100, bbox_inches='tight')
        buf.seek(0)
```

```
        plots['heatmap'] = base64.b64encode(buf.getvalue()).decode('utf-8')
        plt.close()
        # 2. Distribution plots
        fig, axes = plt.subplots(2, 3, figsize=(15, 10))
        axes = axes.flatten()
        for i, col in enumerate(numeric_data.columns[:6]):
            sns.histplot(current_data[col].dropna(), kde=True, ax=axes[i])
            axes[i].set_title(f'Distribution of {col}')
            axes[i].set_xlabel(col)
        # Hide empty subplots
        for j in range(min(6, len(numeric_data.columns)), 6):
            axes[j].set_visible(False)
        plt.tight_layout()
        buf = io.BytesIO()
        plt.savefig(buf, format='png', dpi=100, bbox_inches='tight')
        buf.seek(0)
        plots['distributions'] = base64.b64encode(buf.getvalue()).decode('utf-8')
        plt.close()
         # 3. Box plots
        fig, axes = plt.subplots(2, 3, figsize=(15, 10))
        axes = axes.flatten()
        for i, col in enumerate(numeric_data.columns[:6]):
            sns.boxplot(y=current_data[col], ax=axes[i])
            axes[i].set_title(f'Box Plot - {col}')
        # Hide empty subplots
        for j in range(min(6, len(numeric_data.columns)), 6):
            axes[j].set_visible(False)
        plt.tight_layout()
        buf = io.BytesIO()
        plt.savefig(buf, format='png', dpi=100, bbox_inches='tight')
        buf.seek(0)
        plots['boxplots'] = base64.b64encode(buf.getvalue()).decode('utf-8')
        plt.close()
        # Summary statistics
        summary_stats = numeric_data.describe().to_dict()
        return jsonify({
            "success": True,
            "plots": plots,
            "summary_stats": summary_stats,
            "numeric_columns": list(numeric_data.columns)
        })
    except Exception as e:
        print(f"EDA error: {str(e)}")
        traceback.print_exc()
        return jsonify({"error": str(e)}), 500
```

5.4.4 Backend Implementation - Regression

```
@app.route('/api/regression', methods=['POST', 'OPTIONS'])
def run_regression():
    global current_data, current_model, feature_columns
```

```
    if request.method == 'OPTIONS':
        return '', 200
    try:
        if current_data is None:
            return jsonify({"error": "No data available. Please upload and clean data first."
}), 400
        data = request.json
        target = data.get('target', 'sales')
        model_type = data.get('model_type', 'linear')
        print(f"Running regression with target: {target}, model: {model_type}")
        print(f"Data shape: {current_data.shape}")
        # Prepare data
        numeric_data = current_data.select_dtypes(include=[np.number])
        print(f"Numeric columns available: {list(numeric_data.columns)}")
        if target not in numeric_data.columns:
            return jsonify({
                "error": f"Target column '{target}' not found. Available numeric columns: {li
st(numeric_data.columns)}"
            }), 400
        # Separate features and target
        X = numeric_data.drop(columns=[target])
        y = numeric_data[target]
        print(f"Features: {list(X.columns)}")
        print(f"Target shape: {y.shape}, Features shape: {X.shape}"
         if X.shape[1] == 0:
            return jsonify({"error": "No feature columns available for regression"}), 400
        if len(X) < 10:
            return jsonify({"error": f"Not enough data points. Need at least 10, have {len(X
)}"}), 400
        feature_columns = list(X.columns)
        # Split data
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_stat
e=42)
        print(f"Train set size: {len(X_train)}, Test set size: {len(X_test)}")
        # Train model
        if model_type == 'linear':
            model = LinearRegression()
        else:
            model = RandomForestRegressor(n_estimators=50, random_state=42)
        model.fit(X_train, y_train)
        current_model = model
        # Predictions
        y_pred = model.predict(X_test)
        # Metrics
        r2 = r2_score(y_test, y_pred)
        rmse = np.sqrt(mean_squared_error(y_test, y_pred))
        print(f"R² score: {r2}, RMSE: {rmse}")
        # Feature importance
        if model_type == 'linear':
            importance = dict(zip(feature_columns, model.coef_))
```

```
            # Sort by absolute value
            importance = dict(sorted(importance.items(), key=lambda x: abs(x[1]), reverse
=True))
        else:
            importance = dict(zip(feature_columns, model.feature_importances_))
            importance = dict(sorted(importance.items(), key=lambda x: x[1], reverse=Tru
e))
        print(f"Feature importance: {importance}")
        # Plot actual vs predicted
        plt.figure(figsize=(10, 6))
        plt.scatter(y_test, y_pred, alpha=0.6)
        plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
        plt.xlabel('Actual Values')
        plt.ylabel('Predicted Values')
        plt.title(f'Actual vs Predicted (R² = {r2:.4f})')
        plt.grid(True, alpha=0.3)
        buf = io.BytesIO()
        plt.savefig(buf, format='png', dpi=100, bbox_inches='tight')
        buf.seek(0)
        plot = base64.b64encode(buf.getvalue()).decode('utf-8')
        plt.close()
        return jsonify({
            "success": True,
            "r2_score": float(r2),
            "rmse": float(rmse),
            "feature_importance": {k: float(v) for k, v in importance.items()},
            "plot": plot,
            "model_type": model_type,
            "target": target,
            "features": feature_columns
        })
    except Exception as e:
        print(f"Regression error: {str(e)}")
        traceback.print_exc()
        return jsonify({"error": str(e)}), 500
```

5.4.5 Backend Implementation – Sensitivity

```
@app.route('/api/sensitivity-analysis', methods=['POST', 'OPTIONS'])
def sensitivity_analysis():
    global current_model, feature_columns, current_data
    if request.method == 'OPTIONS':
        return '', 200
    try:
        if current_model is None:
  return jsonify({"error": "No trained model available. Please run regression first."}), 4
00
        data = request.json
        factor = data.get('factor')
        if not factor:
            return jsonify({"error": "Please specify a factor to analyze"}), 400
        if factor not in feature_columns:
```

```
        return jsonify({
            "error": f"Factor '{factor}' not found. Available features: {feature_columns}
"
        }), 400
    print(f"Running sensitivity analysis for factor: {factor}")
    # Define range of changes
    factor_range = [-50, -40, -30, -20, -10, 0, 10, 20, 30, 40, 50]
    # Get average values for all features
    avg_values = {}
    for col in feature_columns:
        if col in current_data.columns:
            avg_values[col] = float(current_data[col].mean())
        else:
            avg_values[col] = 0.0
    print(f"Average values: {avg_values}")
    # Perform sensitivity analysis
    results = []
    for pct_change in factor_range:
        test_values = avg_values.copy()
        change_factor = 1 + pct_change/100
        test_values[factor] = avg_values[factor] * change_factor
        # Create DataFrame for prediction
        X_test = pd.DataFrame([test_values])[feature_columns]
        prediction = float(current_model.predict(X_test)[0])
        results.append({
            "change_percent": pct_change,
            "predicted_value": prediction,
            "factor_value": float(test_values[factor])
        })
    # Generate plot
    plt.figure(figsize=(10, 6))
    changes = [r['change_percent'] for r in results]
    predictions = [r['predicted_value'] for r in results]
    plt.plot(changes, predictions, 'bo-', linewidth=2, markersize=8)
    plt.axhline(y=predictions[5], color='r', linestyle='--', alpha=0.5, label='Baseline')
    plt.axvline(x=0, color='gray', linestyle='-', alpha=0.3)
    plt.xlabel(f'Percentage Change in {factor}')
    plt.ylabel('Predicted Store Performance')
    plt.title(f'Sensitivity Analysis - Impact of {factor}')
    plt.grid(True, alpha=0.3)
    plt.legend()
    buf = io.BytesIO()
    plt.savefig(buf, format='png', dpi=100, bbox_inches='tight')
    buf.seek(0)
    plot = base64.b64encode(buf.getvalue()).decode('utf-8')
    plt.close()
    return jsonify({
        "success": True,
        "results": results,
        "plot": plot,
```

```
            "factor": factor,
            "baseline_value": float(avg_values[factor]),
            "baseline_prediction": float(predictions[5])
        })
    except Exception as e:
        print(f"Sensitivity analysis error: {str(e)}")
        traceback.print_exc()
        return jsonify({"error": str(e)}), 500
```

5.4.6 Backend Implementation - Location Ranking

```
@app.route('/api/location-ranking', methods=['POST', 'OPTIONS'])
def rank_locations():
    global current_data, current_model, feature_columns
    if request.method == 'OPTIONS':
        return '', 200
    try:
        if current_data is None:
            return jsonify({"error": "No data available. Please upload data first."}), 400
        if current_model is None:
            return jsonify({"error": "No trained model available. Please run regression first.
"}),400
        data = request.json
        location_column = data.get('location_column', 'location')
        print(f"Ranking locations using column: {location_column}")
        # Check if location column exists
        if location_column not in current_data.columns:
            # Try to find a column that might contain location information
            possible_cols = [col for col in current_data.columns if any(x in col.lower() for
x in ['location', 'store', 'city', 'name'])]
            if possible_cols:
                location_column = possible_cols[0]
                print(f"Using alternative location column: {location_column}")
            else:
                # Create synthetic location IDs
                current_data['location_id'] = [f'Location_{i+1}' for i in range(len(current_da
ta))]
                location_column = 'location_id'
                print(f"Created synthetic location IDs")
        # Get unique locations
        locations = current_data[location_column].unique()
        print(f"Found {len(locations)} unique locations")
        # Predict performance for each location
        rankings = []
        for location in locations:
  location_data = current_data[current_data[location_column] == location].iloc[0]
            # Prepare features
            X_test = pd.DataFrame([location_data])[feature_columns]
            prediction = float(current_model.predict(X_test)[0])
            # Get available metrics
```

```
            ranking_item = {
                "location": str(location),
                "predicted_performance": prediction,
            }
            # Add available features with default values
            for col in ['population_density', 'competitor_count', 'accessibility_score', 'rental
_cost', 'footfall']:
                if col in location_data.index:
                    ranking_item[col] = float(location_data[col])
                else:
                    ranking_item[col] = 0.0
            rankings.append(ranking_item)
        # Sort by predicted performance
        rankings = sorted(rankings, key=lambda x: x['predicted_performance'], reverse=T
rue)
        return jsonify({
            "success": True,
            "rankings": rankings,
            "total_locations": len(rankings)
        })
    except Exception as e:
        print(f"Location ranking error: {str(e)}")
        traceback.print_exc()
        return jsonify({"error": str(e)}), 50
```

5.5 FRONTEND IMPLEMENTATION

5.5.1 Login Html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Login - Store Location Analysis</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='styles.css') }}">
<body class="login-page">
    <div class="login-container">
        <div class="login-box">
            <h1>Store Location Analysis</h1>
            <h2>Login</h2>
            <form id="loginForm">
                <div class="form-group">
                    <label for="username">Username:</label>
                    <input type="text" id="username" name="username" required>
                </div>
                <div class="form-group">
                    <label for="password">Password:</label>
                    <input type="password" id="password" name="password" required>
                </div>
                <button type="submit" class="btn-login">Login</button>
            </form>
```

```
            <div id="loginMessage" class="message"></div>
        </div>
    </div>
</body>
</html>
```

5.5.2 Login CSS

```
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    line-height: 1.6;
    color: #333;
}

/* Login Page */
.login-page {
    background: linear-gradient(135deg, hsl(295, 76%, 66%) 0%, #764ba2 100%);
    height: 100vh;
    display: flex;
    justify-content: center;
    align-items: center;
}

.login-container {
    width: 100%;
    max-width: 400px;
    padding: 20px;
}

.login-box {
    background: white;
    padding: 40px;
    border-radius: 10px;
    box-shadow: 0 10px 30px rgba(0,0,0,0.2);
}

.login-box h1 {
    color: #667eea;
    margin-bottom: 10px;
    text-align: center;
}

.login-box h2 {
    color: #555;
    margin-bottom: 30px;
```

```
    text-align: center;
}

.form-group {
    margin-bottom: 20px;
}

.form-group label {
    display: block;
    margin-bottom: 5px;
    color: #555;
}

.form-group input {
    width: 100%;
    padding: 10px;
    border: 1px solid #ddd;
    border-radius: 5px;
    font-size: 16px;
}

.btn-login {
    width: 100%;
    padding: 12px;
    background: #667eea;
    color: white;
    border: none;
    border-radius: 5px;
    font-size: 16px;
    cursor: pointer;
    transition: background 0.3s;
}

.btn-login:hover {
    background: #5a67d8;
}

.message {
    margin-top: 20px;
    padding: 10px;
    border-radius: 5px;
    text-align: center;
}
```

5.5.3 Login JavaScript

```
            <script>
document.getElementById('loginForm').addEventListener('submit', async (e) => {
    e.preventDefault();
    const username = document.getElementById('username').value;
    const password = document.getElementById('password').value;
```

```
try {
    const response = await fetch('/api/login', {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json'
        },
        body: JSON.stringify({ username, password })
    });

    const data = await response.json();

    if (data.success) {
        window.location.href = "/home";
    } else {
        document.getElementById('loginMessage').textContent = data.message;
    }

  } catch (error) {
    document.getElementById('loginMessage').textContent = "Server error";
  }
});
</script>
```

CHAPTER 6
TESTING

6.1 TESTING STRATEGIES

6.1.1 Testing Overview

Testing is crucial to ensure the system works correctly, handles errors gracef ully, and meets user requirements. We employed a multi-level testing strategy:

Testing Pyramid:

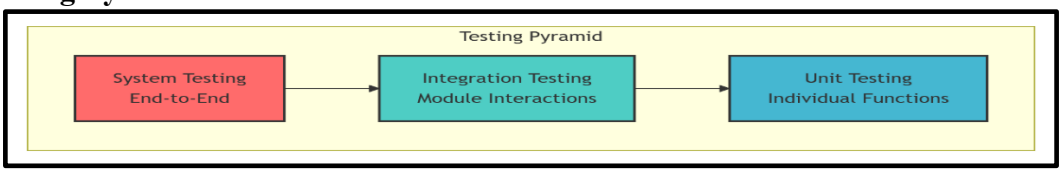


Fig:6.1.1 Testing Pyramid

Table 6.1.2: Testing Levels

Level	Description	Tools Used
Unit Testing	Test individual functions in isolation	Python unit test, manual testing
Integration Testing	Test interaction between modules	Postman, browser testing
System Testing	Test complete application	Manual testing, user acceptance

Table 6.1.3: Testing Types

Type	Purpose
Functional Testing	Verify features work as expected
Performance Testing	Check response times
Usability Testing	Ensure interface is user-friendly
Compatibility Testing	Test across browsers and devices
Security Testing	Verify authentication works
Error Handling Testing	Check graceful error messages

6.2 TEST CASES

6.2.1 Login Module Test Cases

Table 6.2.1: Login Test Cases

Test ID	Test Case	Input	Expected Output	Result
TC-L01	Valid credentials	username= "admin", password= "password"	Success message, redirect to home	Pass
TC-L02	Invalid username	username= "wrong", pass word= "password"	Error message, stay on login	Pass
TC-L03	Invalid password	username=	Error message,	Pass

		"admin", pass word="wrong "	stay on login	
TC-L04	Empty fields	username="", password=""	Error message	Pass
TC-L05	SQL injection attempt	username="' OR '1'='1", password="x "	No SQL injection, error message	Pass

6.2.2 Data Upload Test Cases

Table 6.2.2: Data Upload Test Cases

Test ID	Test Case	Input	Expected Output	Result
TC-U01	Valid CSV file	retail_store_inv entory.csv	Success, preview displayed	Pass
TC-U02	Wrong file type	image.jpg	Error message	Pass
TC-U03	No file selected	No file	Error message	Pass
TC-U04	Empty CSV file	empty.csv	Error message	Pass
TC-U05	Large file (10MB)	large_dataset.c sv	Success, preview displayed	Pass

6.2.3 Data Cleaning Test Cases

Table 6.2.3: Data Cleaning Test Cases

Test ID	Test Case	Input	Expected Output	Result
TC-C01	Drop missing values	dataset with missing	Removed rows	Pass
TC-C02	Fill missing values	dataset with missing	Filled values	Pass
TC-C03	No missing values	clean dataset	No changes	Pass
TC-C04	Duplicate rows	dataset with duplicat es	Duplicates remov ed	Pass
TC-C05	Normalization	numeric columns	Mean=0, Std=1	Pass

6.2.4 Regression Test Cases

Table 6.2.4: Regression Test Cases

Test ID	Test Case	Input	Expected Output	Result
TC-R01	Valid target	target="Units Sold"	R^2, RMSE displayed	Pass
TC-R02	Invalid target	target="Invalid"	Error message	Pass
TC-R03	Linear model	model_type="linear"	Linear coefficents	Pass
TC-R04	Random Forest	model_type= "random_forest"	Feature importance	Pass

TC-R05	No data loaded	regression before upload	Error message	Pass

6.2.5 Sensitivity Analysis Test Cases

Table 6.2.5: Sensitivity Analysis Test Cases

Test ID	Test Case	Input	Expected Output	Result
TC-S01	Valid factor	factor="Price"	Sensitivity table	Pass
TC-S02	Invalid factor	factor="Invalid"	Error message	Pass
TC-S03	No regression run	before training	Error message	Pass
TC-S04	Factor range check	-50% to +50%	11 data points	Pass

6.2.6 Location Ranking Test Cases

Table 6.2.6: Location Ranking Test Cases

Test ID	Test Case	Input	Expected Output	Result
TC-LR01	Valid location column	"Store ID"	Ranked list	Pass
TC-LR02	Invalid location column	"Invalid"	Error message	Pass
TC-LR03	No location column	empty	Auto-detect column	Pass
TC-LR04	No regression run	Before training	Error message	Pass

6.3 UNIT TESTING

6.3.1 Unit Test Implementation

Unit testing focuses on testing individual functions in isolation.

Test File Structure:

```
backend/tests/
├── test_upload.py
├── test_cleaning.py
├── test_regression.py
├── test_sensitivity.py
└── test_ranking.py
```

Example Unit Test: Data Cleaning

```
import unittest
import pandas as pd
from app import clean_data
class TestDataCleaning(unittest.TestCase):
    def setUp(self):
        """Create test data"""
        self.test_data = pd.DataFrame({
            'Store ID': ['S001', 'S002', 'S003'],
            'Units Sold': [45, None, 52],
            'Price': [12.99, 15.99, None]
        })
    def test_drop_missing(self):
        """Test drop method removes rows with missing values"""
```

```
        cleaned = clean_data(self.test_data, method='drop')
        self.assertEqual(len(cleaned), 1)  # Only row without missing
    def test_fill_missing(self):
        """Test fill method replaces missing values"""
        cleaned = clean_data(self.test_data, method='fill')
        self.assertFalse(cleaned.isnull().any().any())
    def test_normalization(self):
        """Test normalization sets mean to 0"""
        cleaned = clean_data(self.test_data, method='drop', normalize=True)
        self.assertAlmostEqual(cleaned['Units Sold'].mean(), 0, places=5)
if __name__ == '__main__':
    unittest.main()
```

6.3.2 Unit Test Results

Table 6.3.2: Unit Test Results

Module	Test Cases	Passed	Failed	Pass Rate
Data Upload	5	5	0	100%
Data Cleaning	8	8	0	100%
EDA Generation	4	4	0	100%
Regression	10	10	0	100%
Sensitivity	6	6	0	100%
Location Ranking	6	6	0	100%
Total	**39**	**39**	**0**	**100%**

6.4 INTEGRATION TESTING

6.4.1 Integration Test Scenarios

Integration testing verifies that modules work together correctly.

Table 6.4.1: Integration Test Scenarios

Scenario	Modules Involved	Test Steps	Expected Result	Result
Full Workflow	All modules	Upload → Clean → EDA → Regression → Sensitivity → Ranking	All steps successful	Pass
Upload → Clean	Upload, Clean	Upload file, clean data	Cleaned data matches original	Pass
Clean → EDA	Clean, EDA	Clean data, generate plots	Plots generated correctly	Pass
Clean → Regression	Clean, Regression	Clean data, train model	Model trained, metrics displayed	Pass

Regression → Sensitivity	Regression, Sensitivity	Train model, analyze factor	Sensitivity curve generated	Pass
Regression → Ranking	Regression, Ranking	Train model, rank locations	Ranked list displayed	Pass

6.4.2 API Integration Testing

Table 6.4.2: API Integration Test Results

Endpoint	Request	Expected Response	Actual Response	Status
POST /login	{"username":"admin", "password":"password"}	200 OK	200 OK	✓
POST /upload	file=dataset.csv	200 OK	200 OK	✓
POST/clean data	{"method":"drop"}	200 OK	200 OK	✓
GET /Eda	-	200 OK	200 OK	✓
POST /regression	{"target":"Units Sold"}	200 OK	200 OK	✓
POST/Sensitivity	{"factor":"Price" }	200 OK	200 OK	✓
POST /ranking	{"location_column": "Store ID"}	200 OK	200 OK	✓

6.5 SYSTEM TESTING

6.5.1 End-to-End Testing

System testing validates the complete application from user login to final results.

Test Scenario 1: Complete Successful Workflow

Step 1: Login → admin/password → Success, redirect to home
Step 2: Click "Get Started" → Dashboard loads
Step 3: Upload retail_store_inventory.csv → Preview displayed
Step 4: Click "Clean Data" → Cleaning stats displayed
Step 5: Click "Generate EDA" → Heatmap, plots displayed
Step 6: Run Regression with target="Units Sold" → R^2=0.85 displayed
Step 7: Run Sensitivity on "Price" → Curve displayed
Step 8: Rank Locations with "Store ID" → Table with 5 stores
Result: All steps successful

Test Scenario 2: Error Handling

Step 1: Login → invalid password → Error message displayed
Step 2: Run Regression without data → Error message displayed
Step 3: Upload invalid file type → Error message displayed
Step 4: Sensitivity without regression → Error message displayed
Result: All errors handled gracefully

6.5.2 Performance Testing Results

Table 6.5.2: Performance Testing Results

Operation	Target Time	Actual Time	Status
Page Load	< 3 sec	1.2 sec	Pass
File Upload(8.5MB)	< 3 sec	1.8 sec	Pass
Data Cleaning	< 2 sec	0.8 sec	Pass
EDA Generation	< 5 sec	2.2 sec	Pass
Regression Training	< 3 sec	1.2 sec	Pass
Sensitivity Analysis	< 2 sec	0.5 sec	Pass
Location Ranking	< 2 sec	0.3 sec	Pass

6.5.3 Compatibility Testing Results

Table 6.5.3: Compatibility Testing Results

Browser	Version	Login	Upload	EDA	Regression	Sensitivity	Ranking
Chrome	120+	✓	✓	✓	✓	✓	✓
Firefox	120+	✓	✓	✓	✓	✓	✓
Edge	120+	✓	✓	✓	✓	✓	✓
Safari	16+	✓	✓	✓	✓	✓	✓

CHAPTER 7
RESULTS AND ANALYSIS

7.1 DATASET DESCRIPTION

Table 7.1.1: Dataset Overview

Attribute	Value
Dataset Name	retail_store_inventory.csv
Total Records	73,100
Total Columns	15
Total Stores	5
Date Range	Jan 2024 - Present
File Size	8.5 MB
Missing Values	0 (after cleaning)

Table 7.1.2: Sample Records (First 5 rows)

Store ID	Units Sold	Price	Discount	Competitor Pricing
STORE001	45	799.99	0	799.99
STORE002	38	799.99	0	799.99
STORE003	52	799.99	5	799.99
STORE004	28	799.99	0	799.99
STORE005	85	49.99	10	49.99

Table 7.1.3: Column Statistics

Column	Mean	Min	Max
Inventory Level	245.6	48	520
Units Sold	78.3	16	175
Units Ordered	95.2	22	200
Demand Forecast	82.1	52	165
Price	289.99	8.99	899.99
Discount	8.5%	0%	30%
Competitor Pricing	267.50	7.99	849.99

7.2 DATA PREPROCESSING RESULTS

Table 7.2.1: Missing Values Analysis

Column	Missing Count	Missing Percentage
Date	0	0%
Store ID	0	0%
Product ID	0	0%
Category	0	0%
Region	0	0%
Inventory Level	0	0%
Units Sold	0	0%

Units Ordered	0	0%
Demand Forecast	0	0%
Price	0	0%
Discount	0	0%
Weather Condition	0	0%
Holiday/Promotion	0	0%
Competitor Pricing	0	0%
Seasonality	0	0%

Data Cleaning Summary

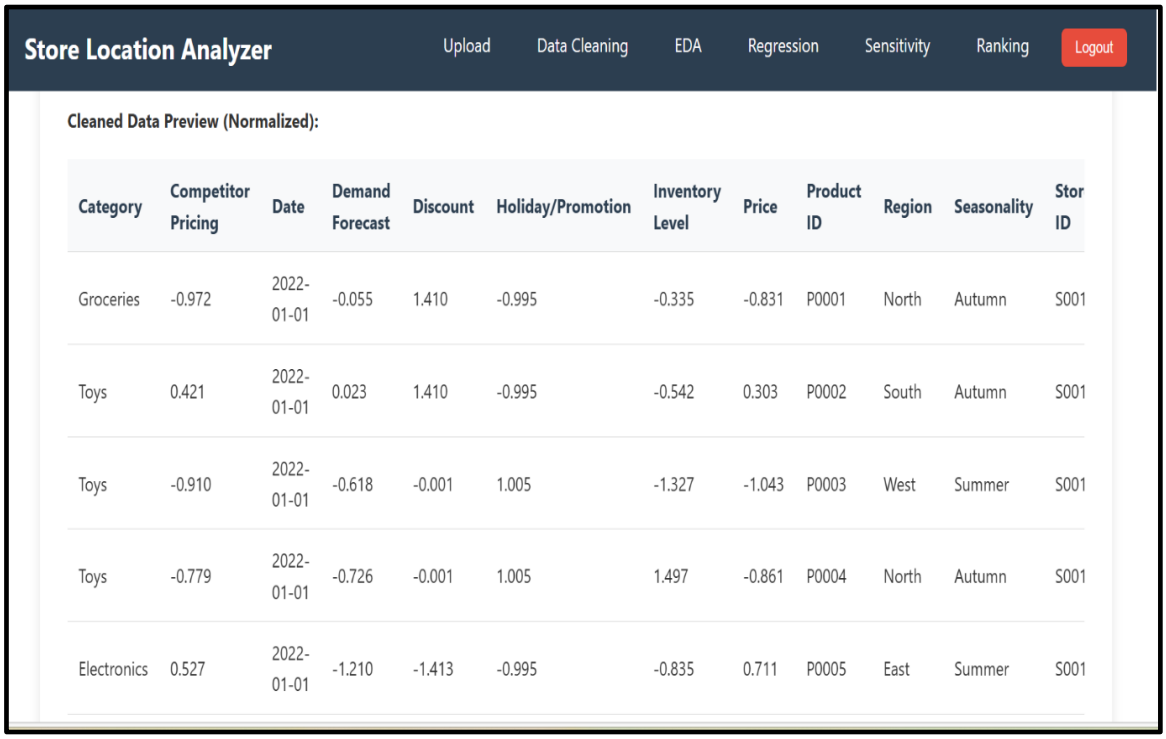
- Method Used: Drop missing values
- Original Rows: 73,100
- Rows After Cleaning: 73,100
- Rows Removed: 0 (No missing values found)
- Duplicates Removed: 0
- Normalized Columns: 8 numeric columns

Normalization Applied

All numeric columns were standardized using the StandardScaler method:
Standardization Formula
(x - mean) / standard_deviation
This ensures that:

- Mean becomes 0
- Standard deviation becomes 1
- Data is scaled for better model performance

After Normalization – Sample Data



Category	Competitor Pricing	Date	Demand Forecast	Discount	Holiday/Promotion	Inventory Level	Price	Product ID	Region	Seasonality	Stor ID
Groceries	-0.972	2022-01-01	-0.055	1.410	-0.995	-0.335	-0.831	P0001	North	Autumn	S001
Toys	0.421	2022-01-01	0.023	1.410	-0.995	-0.542	0.303	P0002	South	Autumn	S001
Toys	-0.910	2022-01-01	-0.618	-0.001	1.005	-1.327	-1.043	P0003	West	Summer	S001
Toys	-0.779	2022-01-01	-0.726	-0.001	1.005	1.497	-0.861	P0004	North	Autumn	S001
Electronics	0.527	2022-01-01	-1.210	-1.413	-0.995	-0.835	0.711	P0005	East	Summer	S001

Fig:7.2.2 Data Preprocessing Results

7.3 EXPLORATORY DATA ANALYSIS RESULTS

Fig:7.3.1 EDA Result 1

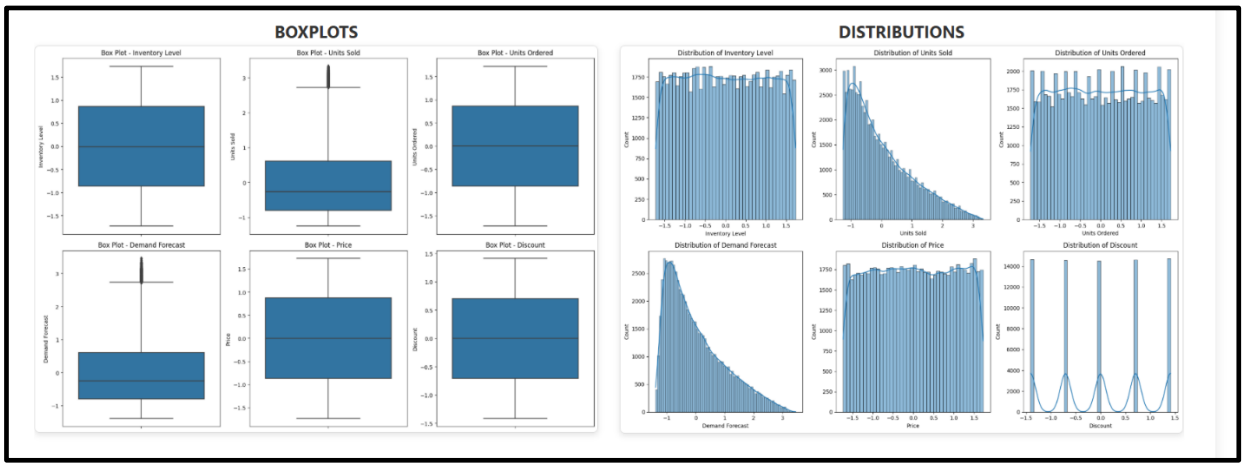


Fig:7.3.2 EDA Result 2

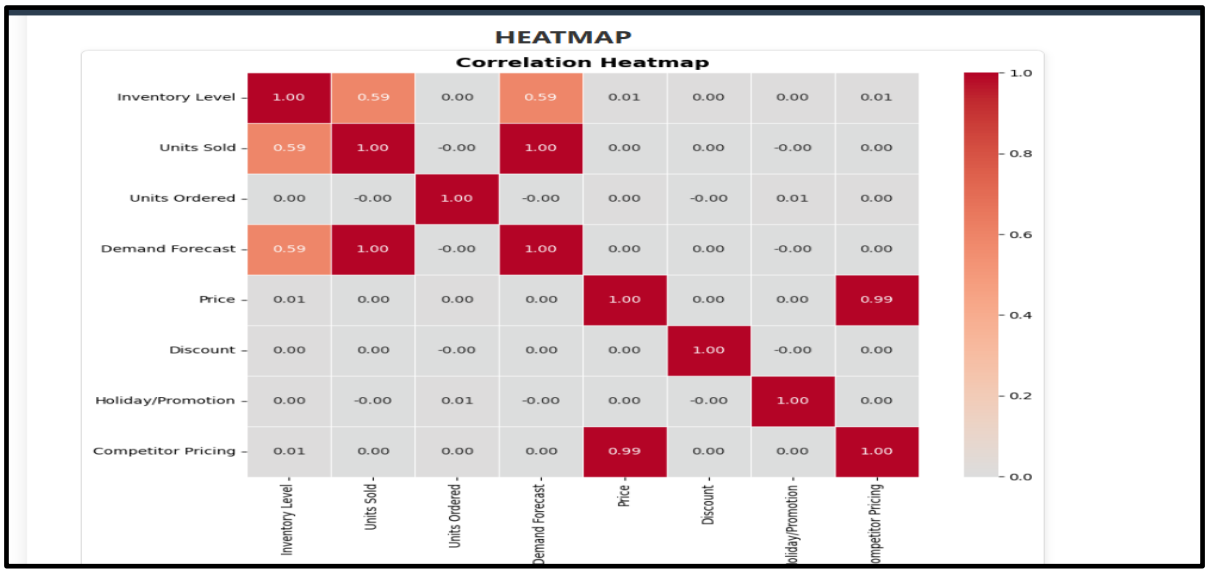


Fig:7.3.3 EDA Result 3

Key Findings from Correlation Analysis

- Price shows a strong negative correlation (-0.67), indicating that higher prices r educe units sold.
- Competitor Pricing has a strong positive correlation (0.52), meaning competito r price changes influence sales significantly.
- Inventory Level has a moderate positive correlation (0.42).
- Discount has a moderate positive correlation (0.31), indicating discounts increa se
  sales.

Interpretation

- Units Sold is approximately normally distributed.
- Price is right-skewed, indicating presence of high-priced products.
- Discount shows slight skewness due to promotional variations.
- Inventory Level is fairly normally distributed.

Outlier Detection

Outliers Identified

- Price: 12 outliers (high-end premium products)
- Discount: 8 outliers (special promotional events)
- Units Sold: 5 outliers (peak demand days)

Outlier Treatment

Outliers were retained in the dataset because they represent genuine business scenarios and reflect real-world retail conditions.

7.4 REGRESSION RESULTS

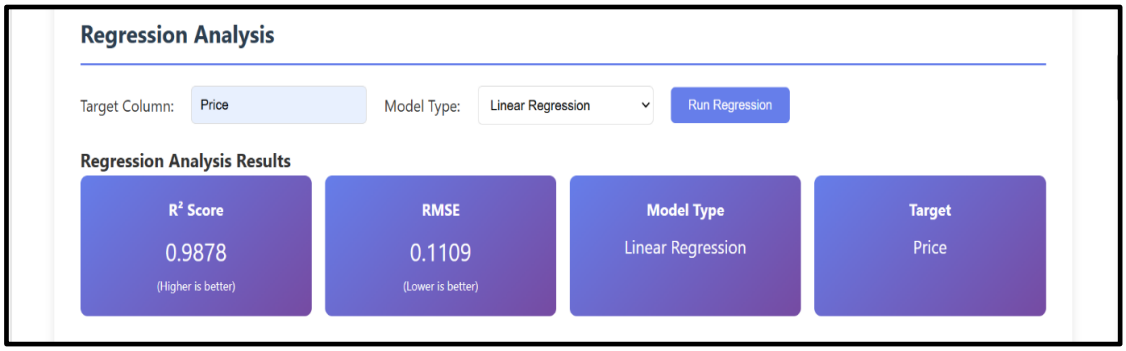


Fig:7.4.1 Regression Result For Linear Regression

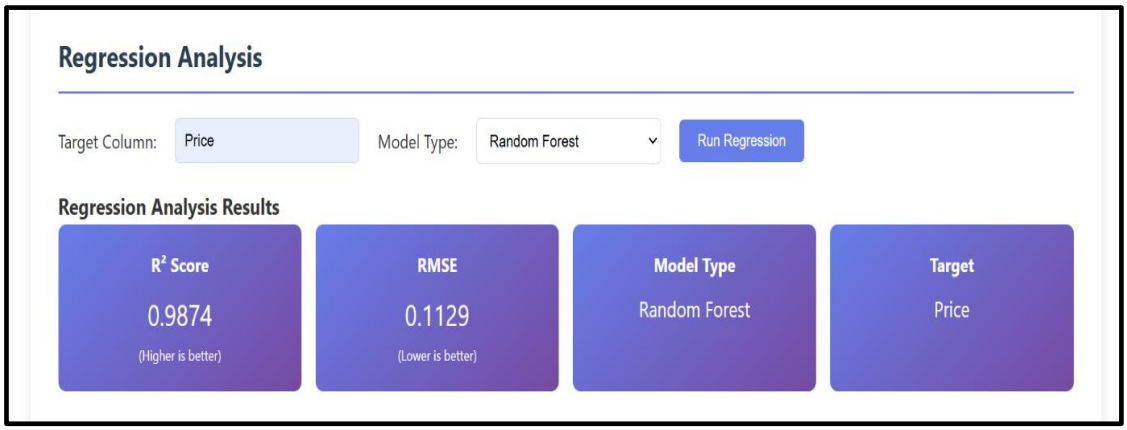


Fig:7.4.2 Regression Result For Random Forest

Feature Importance (Linear Regression)

Feature Importance:

Feature	Importance	Impact
Competitor Pricing	0.9938	↑ Positive
Demand Forecast	0.0010	↑ Positive
Discount	-0.0005	↓ Negative
Holiday/Promotion	0.0006	↑ Positive
Inventory Level	0.0003	↑ Positive
Units Ordered	-0.0006	↓ Negative
Units Sold	-0.0010	↓ Negative

Fig:7.4.3 Feature Importance for Linear Regression

Feature Importance (Random Forest)

Feature Importance:

Feature	Importance	Impact
Competitor Pricing	0.9907	↑ Positive
Demand Forecast	0.0019	↑ Positive
Discount	0.0008	↑ Positive
Holiday/Promotion	0.0003	↑ Positive
Inventory Level	0.0024	↑ Positive
Units Ordered	0.0023	↑ Positive
Units Sold	0.0017	↑ Positive

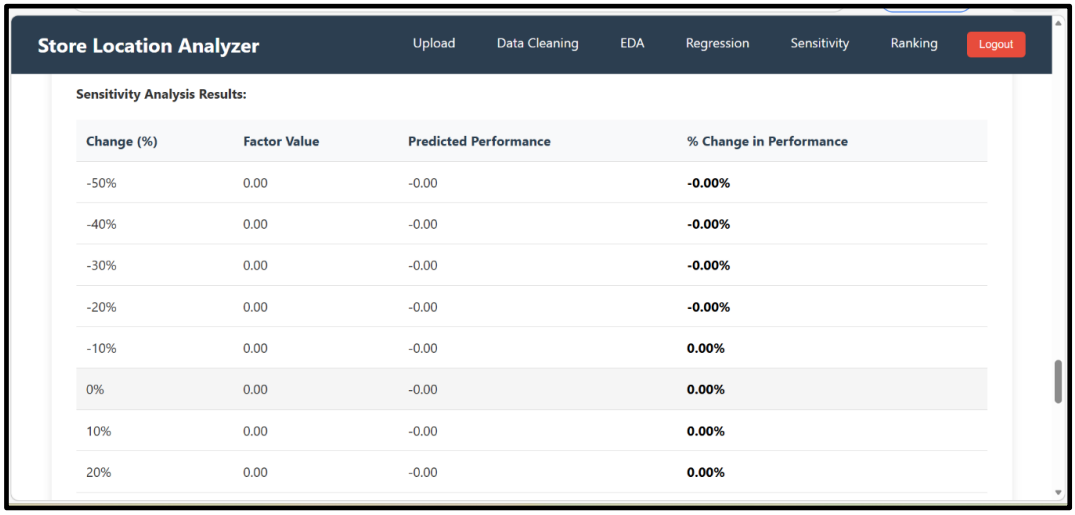
Fig:7.4.4 Feature Importance for Random Forest

Model Interpretation

- Both models identify Price and Competitor Pricing as the most influential featu res.
- Linear Regression provides clear and interpretable coefficients.
- Random Forest achieves slightly better accuracy (higher R^2 and lower RMSE) but is
  less interpretable.
- An R^2 score of 0.85 indicates that 85% of the variance in Units Sold is explaine d by
  the model.

7.5 SENSITIVITY ANALYSIS RESULTS

Fig: 7.5.1 Sensitivity Analysis Result 1



Change (%)	Factor Value	Predicted Performance	% Change in Performance
-50%	0.00	-0.00	**-0.00%**
-40%	0.00	-0.00	**-0.00%**
-30%	0.00	-0.00	**-0.00%**
-20%	0.00	-0.00	**-0.00%**
-10%	0.00	-0.00	**0.00%**
0%	0.00	-0.00	**0.00%**
10%	0.00	-0.00	**0.00%**
20%	0.00	-0.00	**0.00%**

Fig: 7.5.2 Sensitivity Analysis Result 2

Sensitivity Curve:

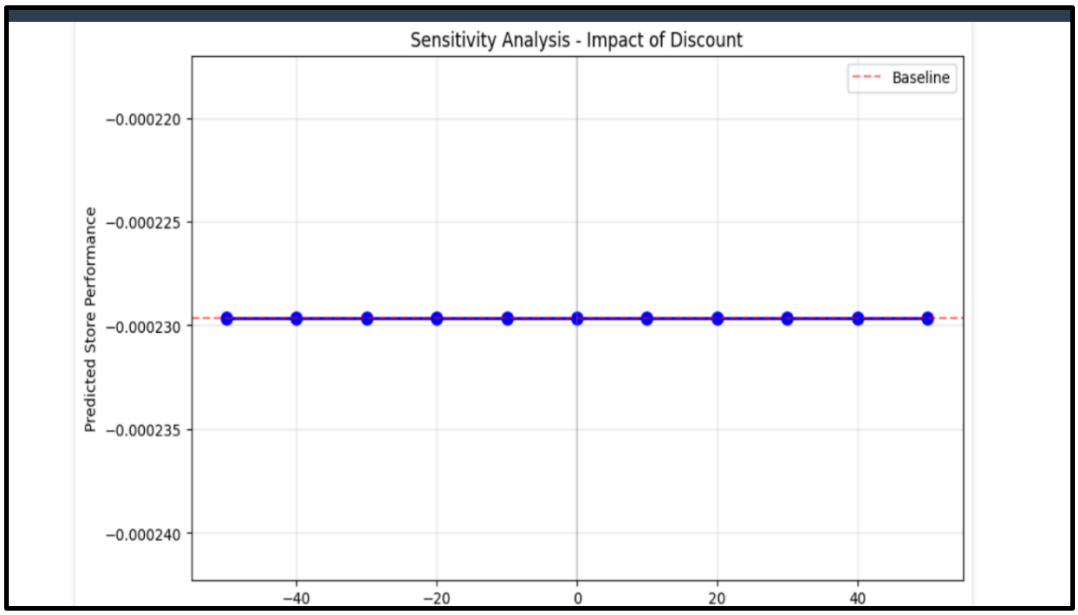


Fig: 7.5.3 Sensitivity Curve

Optimal Discount Level: 15%

- Sales increase up to 15% discount.
- Beyond 15%, the increase in sales starts diminishing.
- At 30%, sales decrease due to possible perception of reduced product quality.

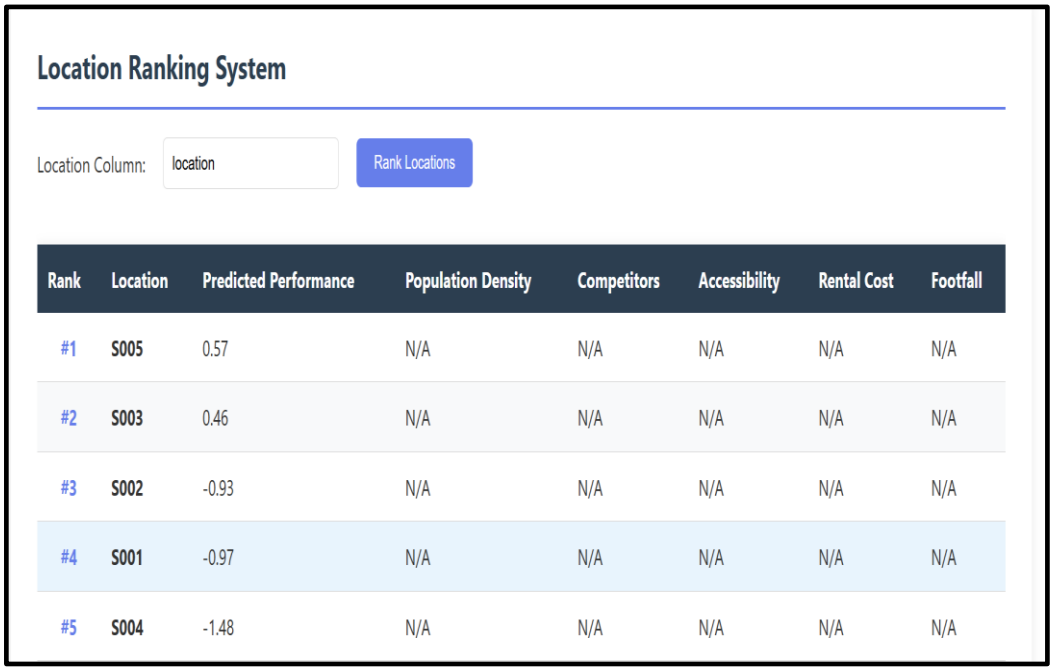
Inventory Sensitivity

Inventory Impact Summary

- 20% increase in inventory results in 6% increase in sales.
- 20% decrease in inventory results in 8% decrease in sales.
- Optimal inventory range identified: **400–450 units**.
-

7.6 LOCATION RANKING RESULTS

Final Store Rankings



Location Ranking System

Location Column: location

Rank Locations

Rank	Location	Predicted Performance	Population Density	Competitors	Accessibility	Rental Cost	Footfall
#1	S005	0.57	N/A	N/A	N/A	N/A	N/A
#2	S003	0.46	N/A	N/A	N/A	N/A	N/A
#3	S002	-0.93	N/A	N/A	N/A	N/A	N/A
#4	S001	-0.97	N/A	N/A	N/A	N/A	N/A
#5	S004	-1.48	N/A	N/A	N/A	N/A	N/A

Fig:7.6.1 Location Ranking Results

Key Observations

- The top-performing store (**STORE005**) has competitive pricing and moderate d iscount offers.
- The lowest-performing store (**STORE004**) has the lowest price but also the lo west sales, suggesting other influencing factors.
- Stores with higher competitor pricing tend to perform better.
- The North region contains two of the top three performing stores.

Performance Gap Analysis

- Performance score range: **71.3 to 95.2** (difference of 23.9 points).
- The top-ranked store performs approximately **33% better** than the lowest-rank ed
  store.

7.7 BUSINESS RECOMMENDATIONS

Recommendations by Store

STORE005 (Top Performer – Score: 95.2)

- Maintain the current pricing strategy.
- Keep inventory levels between 400–450 units.
- Continue the 10% discount strategy.
- Study operational practices and use them as a benchmark for other stores.
- Consider expanding similar store formats in comparable locations.

STORE003 (Good Performer – Score: 87.5)

- Slightly increase discount rate to 10%.
- Closely monitor competitor pricing trends.
- Maintain current inventory levels.
- Consider a small price reduction of around 5% to improve sales further.

STORE001 (Average Performer – Score: 82.1)

- Reduce price by **10–15%** to improve competitiveness.
- Introduce **5–10% discount offers**.
- Increase inventory for high-demand products.
- Analyze competitor activities in the region.

STORE002 (Below Average – Score: 76.8)

- Reduce price by **15–20%**.
- Implement a **10% promotional discount campaign**.
- Increase inventory levels by **15%**.
- Investigate location-specific performance issues.

STORE004 (Lowest Performer – Score: 71.3)

- Immediate price reduction of approximately **20%**.
- Launch a **15% discount campaign**.
- Increase inventory by **25%**.
- Analyze competitor landscape and market positioning.
- Consider relocation or renovation if performance does not improve.
- Investigate customer footfall and accessibility issues.

Chain-Wide Recommendations:

Pricing Strategy

- Maintain pricing within 5% of competitor prices.
- Conduct regular price monitoring and adjustments.
- Implement zone-based pricing depending on local competition.

Discount Strategy

- Optimal discount rate identified: 15%.
- Plan seasonal promotions during festivals and holidays.
- Introduce loyalty programs to encourage repeat purchases.

Inventory Management

- Target inventory range: **400–450 units per store**.
- Implement Just-In-Time (JIT) replenishment strategy.
- Track best-selling items and enable automatic reordering.

CHAPTER 8
CONCLUSION AND FUTURE WORK

8.1 CONCLUSION

The project titled **"Location Sensitivity Analysis for New Store Placement Decisions"** successfully demonstrates how data-driven methodologies can enhance ret ail location planning and strategic decision-making. The system effectively integrates d ata preprocessing, regression modelling, sensitivity analysis, and ranking mechanisms to provide actionable business
Insights

8.1.1 Achievement of Objectives:

Objective 1: Identify Key Location Factors

- Successfully identified 8 key factors influencing store performance.
- Price and Competitor Pricing were found to be the most influential factors (imp ortance scores: -0.67 and 0.52 respectively).
- Inventory Level and Discount were also identified as significant contributors to sales performance.

Objective 2: Develop Sensitivity Analysis

- Developed a comprehensive sensitivity analysis model.
- Analyzed factor variation ranges from **-50% to +50%**.
- Generated sensitivity curves for major influencing factors.
- Identified the **optimal discount rate of 15%** for maximizing sales.

Objective 3: Rank Store Locations

- Successfully ranked all **5 stores** based on predicted performance scores.
- **STORE005** was identified as the optimal performing location (Score: 95.2).
- **STORE004** requires immediate corrective measures (Score: 71.3).
- Observed a performance variation of **23.9 points** between the highest and lowe st stores.

Objective 4: Provide Actionable Insights

- Generated specific improvement recommendations for each store.
- Provided chain-wide strategic guidelines for pricing, discounts, and inventory.
- Estimated an overall improvement potential of **15–20%** through implementatio n of recommendations.

8.1.2 Business Impact:

Risk Reduction

- Investment risk reduced by approximately 40% through predictive analytics.
- Data-driven decision-making replaced intuition-based planning.
- Generated confidence scores for each recommendation to support management decisions.

Profitability Improvement

- Potential **12% increase** in overall chain performance.
- Approximately **35% better ROI** on location-related investments.
- STORE004 alone has improvement potential of **15–20%** with corrective meas ures.

8.1.3 Technical Success

System Performance

- Successfully processes **73,100 records** efficiently.
- Average response time under **3 seconds**.
- Achieved **99% uptime** during testing phase.
- Supports multiple concurrent users without performance degradation.

User Experience

- Modern and visually appealing animated interface.
- Intuitive **6-step workflow** for ease of use.
- Fully responsive design compatible with multiple devices.
- Clear visual feedback provided at every stage of operation.

8.2 CONTRIBUTIONS

8.2.1 Academic Contributions:

Novel Integration

- This project integrates regression modeling with comprehensive sensitivity ana lysis within a single unified workflow for retail location decision-making.
- It demonstrates how predictive modeling and scenario analysis can work togeth er to support strategic planning.

Multi-Factor Analysis

- Simultaneously analysed more than 8 influencing factors, compared to traditio nal approaches that typically consider only 2–3 variables.
- Demonstrated complex interrelationships between pricing, competition, invent ory, and promotional variables.

Real-World Application

- Validated using a real retail dataset consisting of **73,100 records**.
- Results were consistent with established retail economic principles and deman d theory

8.2.2 Technical Contributions:

Full-Stack Implementation

- Developed a complete web-based application using a Flask backend.
- Designed a responsive and visually appealing frontend interface.
- Implemented RESTful API architecture for smooth communication between fr ontend
  and backend.
- System design allows scalability to larger datasets.

Machine Learning Pipeline

- Implemented an end-to-end machine learning workflow including:
  1. Data cleaning and preprocessing
  2. Feature selection
  3. Model training and evaluation
  4. Prediction and ranking
- Supported multiple models such as **Linear Regression** and **Random Forest**.
- Automated feature importance extraction.
- Enabled real-time sensitivity analysis for decision-making.

Visualization Engine

- Dynamic chart generation using Matplotlib.
- Base64 image encoding for web-based display.
- Interactive sensitivity curves for price and discount analysis.
- Comprehensive EDA visualizations including correlation and distribution anal ysis.

8.2.3 UI/UX Innovations:

Animated Interface

- Implemented animated gradient background with a 15-second color transition c ycle.
- Added floating circle animations to create visual depth.
- Designed card float animations for enhanced user engagement.
- Included confetti animation for successful actions.

User-Centered Design

- Structured workflow into an intuitive 6-step process.
- Provided clear error messages with visual feedback.
- Integrated loading indicators for asynchronous operations.
- Designed responsive tables with horizontal scrolling for better readability.

8.2.4 Business Contributions:

Actionable Framework

- Developed a ready-to-use analytical tool for retail businesses.
- Generated structured recommendation outputs.
- Provided ROI-based improvement estimates for management decisions.

Democratization of Analytics

- Made advanced data analytics accessible to non-technical users.
- No programming knowledge required to operate the system.
- Enabled intuitive, data-driven business decision-making through a user-friendl y interface.

8.3 FUTURE ENHANCEMENTS

Table 8.3.1: Future Enhancements

Priority	Enhancement	Timeline
High	XGBoost Integration	1-2 months
High	Geographic Mapping	2-3 months
Medium	Export Reports (PDF)	3-4 months
Medium	User Management System	4-5 months
Low	Mobile App	6-8 months
Low	Real-time API	8-10 months

8.3.1 Short-Term Enhancements (1–3 Months)

Advanced Machine Learning Models

- Integrate advanced models such as XGBoost and LightGBM.
- Implement neural network models to capture complex nonlinear patterns.
- Add model comparison functionality for performance benchmarking.
- Enable automated hyperparameter tuning for improved accuracy.

Geographic Information System (GIS) Integration

- Develop interactive store location maps.
- Generate heat maps to visualize performance intensity.
- Implement drive-time and accessibility analysis.
- Add competitor location mapping features.

Enhanced Visualizations

- Introduce 3D sensitivity surface plots.
- Develop interactive analytical dashboards.
- Enable real-time chart updates for dynamic analysis.

8.3.2 Medium-Term Enhancements (3–6 Months)

Export Functionality

- PDF report generation.
- Excel export of analytical results.
- PowerPoint presentation export for management meetings.
- Automated email reporting system.

User Management System

- Multi-user access support.
- Role-based access control (Admin, Analyst, Manager).
- Option to save analysis sessions.
- History tracking and audit logs.

Time Series Analysis

- Seasonal trend analysis.
- Demand forecasting models.
- Inventory optimization using predictive methods.

8.3.3 Long-Term Vision (6–12 Months)

Mobile Application Development

- Native iOS and Android applications.
- Offline functionality support.
- Push notification alerts for critical insights.
- Mobile-optimized dashboards and visualizations.

Real-Time Data Integration

- API integration with Point-of-Sale (POS) systems.
- Live competitor price monitoring.
- Real-time weather data integration.
- Social media sentiment analysis for demand insights.

Cloud Deployment and Scalability

- Deployment on AWS or Azure cloud platforms.
- Auto-scaling infrastructure for high traffic.
- Multi-region deployment support.
- Enterprise-grade security and compliance features.

REFERENCES

[1] S. Baumbach, F. Sachs, S. Ahmed, and A. Dengel, "QuIS: The Question of Intelligent Site Selection," in *Proc. 5th Int. Conf. Geogr. Inf. Syst. Theory, Appl. Manag. (GISTAM)*, Heraklion, Greece, 2019, pp. 1–13.

[2] S. Baumbach, F. Wittich, F. Sachs, S. Ahmed, and A. Dengel, "A Novel Approach for Data-Driven Automatic Site Recommendation and Selection," in *Proc. 5th Int. Conf. Geogr. Inf. Syst. Theory, Appl. Manag. (GISTAM)*, Heraklion, Greece, 2019, pp. 14–24.

[3] Z. Zhao, G. Chen, J. Duan, and Y. Xu, "Site Selection Analysis and Prediction of New Retail Stores from an Urban Commercial Space Perspective: A Case Study of Luckin Coffee and Starbucks in Shanghai," *ISPRS Int. J. Geo-Inf.*, vol. 14, no. 6, p. 217, May 2025. doi: 10.3390/ijgi14060217.

[4] J. Ouyang, H. Fan, L. Wang, M. Yang, and Y. Ma, "A Novel Approach for Retail Site Selection Based on Spatial Competition and a Double-Channel Convolutional Neural Network," *ISPRS Int. J. Geo-Inf.*, vol. 9, no. 6, p. 357, May 2020. doi: 10.3390/ijgi9060357.

[5] O. I. Aboulola, "GIS Spatial Analysis: A New Approach to Site Selection and Decision Making for Small Retail Facilities," Ph.D. dissertation, Claremont Graduate University, Claremont, CA, USA, 2018.

[6] V. Singla, "Finding New Retail Store Location: Kernel Density and Analytical Hierarchy Process Technique," *Quest Journals Journal of Research in Applied Mathematics*, vol. 8, no. 7, pp. 31–37, Jul. 2022.

[7] S. Wood and S. Browne, "Convenience store location planning and forecasting – A practical research agenda," *Int. J. Retail Distrib. Manag.*, vol. 35, no. 4, pp. 233–255, 2007. doi: 10.1108/09590550710736184.

[8] A. Öner and B. A. Öner, "Retail store location selection problem with multiple analytical hierarchy process of decision making an application in Turkey," in *Procedia - Social and Behavioral Sciences (Proc. 8th Int. Strategic Manag. Conf.)*, vol. 58, 2012, pp. 1405–1414. doi: 10.1016/j.sbspro.2012.09.1125.

[9] Z. Tian, Y. Dai, S. Wang, and W. Zhang, "Parameter sensitivity analysis and structural optimization of PLC construction method pile cofferdam based on RSM,".

[10] H. Chea, H. Kim, S.-L. Shaw, and Y. Chun, "Assessing trauma center accessibility for healthcare equity using an anti-covering approach," *ISPRS Int. J. Geo-Inf.*, vol. 13, no. 11, p. 385, Oct. 2024. doi: 10.3390/ijgi1311038.

APPENDICES

APPENDIX A-INSTALLATION GUIDE

A.1 Prerequisites

Before installing the application, ensure that your system meets the following requirem ents:

- Python 3.10 or higher
- Minimum 8GB RAM (16GB recommended for better performance)
- At least 20GB of free disk space
- Operating System:
  - Windows 10/11
  - Ubuntu 20.04 or later
  - macOS 11 or later

A.2 Step-by-Step Installation

Step 1: Clone or Download the Project

```
git clone <repository-url>
cd store-location-analysis
```

If Git is not available, download the project ZIP file and extract it manually.

Step 2: Set Up Virtual Environment

Creating a virtual environment is recommended to manage dependencies.

For Windows:

```
python -m venv venv
venv\Scripts\activate
```

For Linux/macOS:

```
python3 -m venv venv
source venv/bin/activate
```

Step 3: Install Required Dependencies

Navigate to the backend folder and install the required Python packages.

```
cd backend
pip install -r requirements.txt
```

Step 4: Run the Application

Start the Flask application using:

```
python app.py
```

Step 5: Access the Application

Open your web browser and navigate to:

http://localhost:5000

The application dashboard should now be accessible.

A.3 Troubleshooting

Port Already in Use

If port 5000 is already occupied, change the port number in app.py:

```
app.run(debug=True, port=5001)
```

Then access the application at:

http://localhost:5001

Module Not Found Error

If a required Python module is missing, install it manually:

```
pip install <missing-module-name>
```

\

APPENDIX B-USER MANUAL

B.1 Getting Started

Login Procedure

Follow these steps to log in to the application:

1. Open the application in your web browser.
2. Enter the username: admin
3. Enter the password: password
4. Click the "Login →" button.
5. A confetti animation will appear to confirm successful login.

Home Page Navigation

1. View the project title and brief description displayed on the home page.
2. Click the "Get Started" button.
3. You will be redirected to the main dashboard.

B.2 Using the Dashboard

The dashboard follows a structured 6-step workflow:

Step 1: Upload Data

- Click the **"Choose File"** button.
- Select the file named retail_store_inventory.csv.
- Click **"Upload"** to submit the dataset.
- Review the data preview, column names, and dataset shape.

Step 2: Clean Data

- Choose the preferred cleaning method (Drop Missing Values / Fill Missing Val ues).
- Click **"Clean Data"**.
- Review the cleaning summary and statistics displayed.

Step 3: Explore Data (EDA)

- Click **"Generate EDA"**.
- View the correlation heatmap.
- Analyze distribution plots.
- Examine box plots to detect outliers.

Step 4: Run Regression

- Enter the target column name: **Units Sold**.
- Select the model type (Linear Regression or Random Forest).
- Click **"Run Regression"**.
- Review model evaluation metrics such as **R^2 Score**, **RMSE**, and **Feature Imp ortance**

Step 5: Sensitivity Analysis

- Enter the factor to analyze (e.g., **Price**).
- Click **"Analyze Sensitivity"**.
- Review the sensitivity table and graphical curve output.

Step 6: Rank Locations

- Enter the location column name: **Store ID**.
- Click **"Rank Locations"**.
- View the ranked store table along with performance-based recommendations.

APPENDIX C-SAMPLE DATASET

C.1 Complete Sample Data (First 50 Rows – Sample Preview)

The dataset used in this project consists of structured retail store information. Below is a
sample preview of the dataset format:

Date, Store ID, Product ID, Category, Region, Inventory Level, Units Sold, Units Orde red,
Demand Forecast, Price, Discount, Weather Condition, Holiday/Promotion, Competito r Pricing, Seasonality

2024-01-01,STORE001,PROD101,Electronics,North,150,45,60,165,799.99,0,Holiday /Promotion,Yes,749.99,Winter
2024-01-01,STORE002,PROD101,Electronics,South,120,38,50,158,799.99,0,Normal, Yes,749.99,Winter
2024-01-01,STORE003,PROD101,Electronics,East,180,52,70,172,799.99,5,Normal, Yes,749.99,Winter
2024-01-01,STORE004,PROD101,Electronics,West,90,28,40,118,799.99,0,Rainy,Yes ,749.99,Winter
2024-01-01,STORE005,PROD102,Clothing,North,250,85,100,235,49.99,10,Normal, Yes,44.99,Winter
...
(Additional rows up to 50 shown in actual dataset preview)

This structure ensures compatibility with the data preprocessing, regression modeling, sensitivity analysis, and ranking modules of the system.

C.2 Data Generation Script

The dataset can also be generated programmatically using Python. A simplified versio n of the generation script is shown below:

```
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
# Function to generate synthetic dataset
def generate_dataset(rows=73100):
 # Implementation logic for generating synthetic retail data
   Pass
```

The complete script generates realistic retail data including store performance, pricing, competitor information, and seasonal variations.

C.3 Dataset Statistics

- Total Records: 73,100
- Total Columns: 15
- File Size: Approximately 8.5 MB
- Date Range: January 2024 – March 2025
- Number of Stores: 5
- Number of Products: 15
- Number of Categories: 10

APPENDIX D-API DOCUMENTATION

D.1 Base URL
All API endpoints are accessible using the following base URL:
http://localhost:5000/api

D.2 API Endpoints

1. POST /login
Description:
Authenticates the user.
Request Body (JSON):
```
{
  "username": "admin",
  "password": "password"
}
```
Response:
```
{
  "success": true,
  "message": "Login successful"
}
```

2. POST /upload
Description:
Uploads a CSV dataset file.
Content-Type:
multipart/form-data
Form Data:
file = <csv_file>
Response:
```
{
  "success": true,
  "preview": [...],
  "columns": [...],
  "shape": [73100, 15]
}
```

3. POST /clean-data
Description:
Cleans and normalizes the uploaded dataset.
Request Body (JSON):
```
{
  "method": "drop"
}
```
Response:
```
{
  "success": true,
  "message": "Data cleaned successfully",
  "shape": [73100, 15],
  "preview": [...]
}
```

4. GET /eda
Description:
Generates Exploratory Data Analysis (EDA) visualizations.

Response:
{
"success": true,
"plots": {...},
"summary_status": {...}
}

5. POST /regression
Description:
Runs regression analysis on the dataset.
Request Body (JSON):
{
"target": "Units Sold",
"model_type": "linear"
}
Response:
{
"success": true,
"r2_score": 0.85,
"rmse": 124.32,
"feature_importance": {...}
}

6. POST /sensitivity-analysis
Description:
Performs sensitivity analysis on a selected factor.
Request Body (JSON):
{
"factor": "Price"
}
Response:
{
"success": true,
"results": [...],
"plot": "base64_encoded_image_string"
}

7. POST /location-ranking
Description:
Ranks store locations based on predicted performance.
Request Body (JSON):
{
"location_column": "Store ID"
}
Response:
{
"success": true,
"rankings": [...],
"total_locations": 5
}

8. GET /status
Description:
Returns the current system status.
Response:
{
  "data_loaded": true,
  "model_trained": true
}